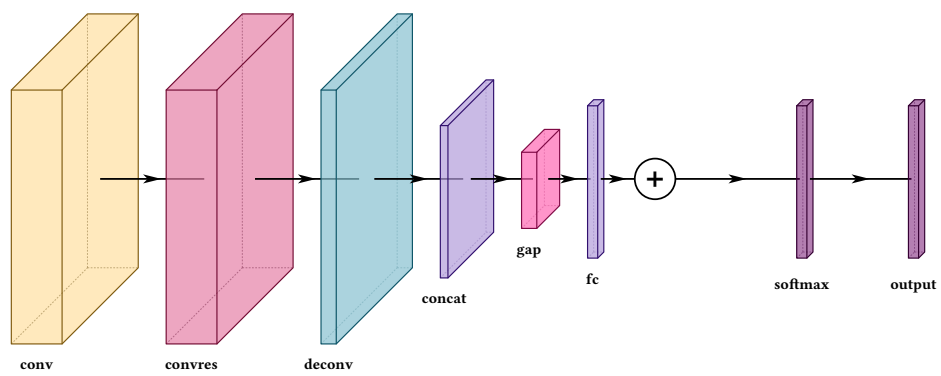


VINNT

Vision Illustration of Neural Networks in Typst



Version 0.1.0 · J.C. Vaught

Every figure in this manual is compiled from the code printed beside it.

Contents

1	What this is	3
1.1	How this manual is organised	3
1.2	Installing	3
1.3	The shape of a figure	3
2	Writing a layer	4
2.1	Options that do not exist are errors	4
2.2	Naming a layer	4
2.3	The list is just an array	5
3	Layer types	7
3.1	The convolution family	7
3.2	Pooling attaches to the block in front	8
3.3	The sum node	9
3.4	Pooling to a vector, and joining	10
3.5	Classifier heads	10
3.6	The input layer	11
3.7	Putting the types together	11
4	Sizing a block	13
4.1	Thickness	13
4.2	Flat layers	15
4.3	Sizing from the tensor shape	15
4.4	By hand against by shape	17
5	Spacing	18
5.1	Why a constant offset fails	18
5.2	Spacing by hand	19
5.3	The white space constant	20
6	Labels	21
6.1	Channel numbers	21
6.2	Axis names	22
6.3	Labelling the arrow between two blocks	23
6.4	When labels collide	23
6.5	The placement options	25
7	Images inside blocks	27
7.1	Content that is not an image	28
8	Colour	30
8.1	Activation bands	30
8.2	Palettes	31
9	Blocks of your own	33
9.1	Building a vocabulary	34
10	Connections	36
10.1	Routing modes	36
10.2	Lane heights	38
10.3	Where a route arrives	40
10.4	Fanning several routes into one block	41
10.5	Telling routes apart	43
10.6	Routes in the legend	43
10.7	Labelling a route	44
10.8	Suppressing an axis arrow	44
10.9	Two composites	45
11	Groups and repeats	48

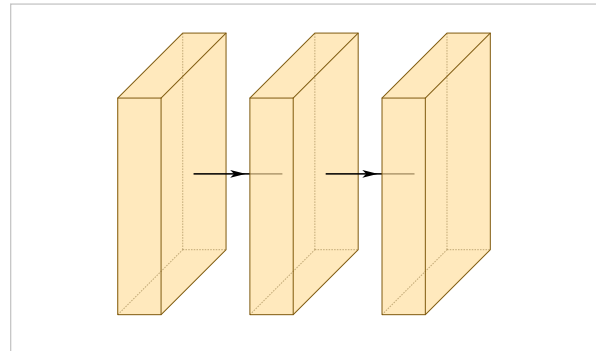
11.1	Tinting and stacking	49
11.2	Repeated blocks	50
11.3	A composite	52
12	Parallel branches	53
12.1	Separation and count	53
12.2	Stacking along the projection	55
12.3	Open at one end	57
12.4	Nesting	57
12.5	Branches are not a separate world	58
12.6	Two composites	60
13	The figure as a whole	62
13.1	Legends	62
13.2	Fitting the page	63
13.3	Line weight	64
13.4	The projection	64
14	Patterns	66
14.1	A classifier	66
14.2	An encoder-decoder	66
14.3	Residual stages	67
14.4	A multi-scale head	68
14.5	A repeated pair	69
15	Every option	71
15.1	Options by layer type	71
15.2	Connection options	71
15.3	Group options	72
15.4	Arguments to draw-network	72
15.5	Also exported	72
15.6	Coverage	72
16	Errors	73
17	Where to go next	74

What this is

VINNT draws layered neural network architectures as isometric block diagrams, the kind that open a paper's method section. It is a Typst package built on [CeTZ](#).

You describe the network as an array of layers. The package works out the geometry, the spacing, the arrows between blocks, the routing of any skip connection you add, and the legend.

```
#draw-network((
  conv(),
  conv(),
  conv(),
))
```



Three convolutions, no options at all. Sizes, spacing and arrows are defaults.

1.1 How this manual is organised

Every option is shown on its own, usually across several figures, because one picture of an option tells you it exists and three tell you what it does. Where an option has a failure mode, the failure is shown next to the fix. Each chapter ends with a composite figure built from what the chapter covered, and chapter 15 is nothing but composites.

Complete published architectures — AlexNet, VGG, ResNet, U-Net, FCN-8, YOLO26-n and six RGB-IR fusion variants — are not here. They are whole readable files in `examples/` in the repository, and they are better read as files than quoted as fragments.

1.2 Installing

```
#import "@preview/vinnt:0.1.0": draw-network, conv, pool, input
```

Every example in this manual assumes an import like that one. Import `draw-network` plus whichever constructors you use, or take everything with `: *`, which is what the examples here do.

VINNT is not yet published on Typst Universe. Until it is, point the import at a local copy: `#import "path/to/src/lib.typ": *`, which is what every example in this manual does.

1.3 The shape of a figure

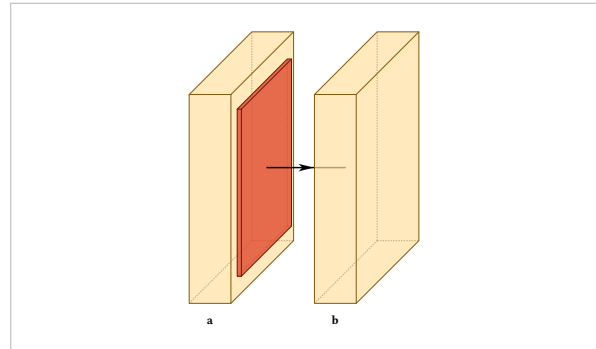
```
#draw-network(
  layers,           // an array: the only positional argument
  connections: (),  // routes between named layers
  groups: (),       // labelled brackets under named layers
  ...              // figure-wide settings, chapter 14
)
```

Everything in this manual is either an option on a layer, an option on a connection or a group, or one of those figure-wide settings.

Writing a layer

A layer is a constructor call. Every type has one, named after it, and its arguments are the options:

```
#draw-network((
  conv(label: "a"),
  pool(),
  conv(label: "b"),
))
```



A constructor's signature is exactly the set of options that type accepts, so an editor can list them while you type, and Typst rejects a misspelled argument by name before the package ever sees it. A plain dictionary in the layer list is an error rather than a second way to write the same thing, so every figure reads the same. When options arrive as a dictionary anyway — shared settings, an imported dump — spread it into the constructor: `conv(..opts)`.

2.1 Options that do not exist are errors

An option a layer type does not read is rejected, not ignored:

```
error: unknown layer option "hieght" on layer 3 (type "conv").
Did you mean "height"? Options accepted here: bandfill, channels,
connection-label, depth, fill, height, image, label, ...
```

This matters more than it sounds. A key that is quietly ignored produces a figure that is merely wrong, and a wrong figure reads as the package being broken rather than as the typo it is. The check covers layer options, connection options, group options, unknown layer types, a layer with no type, and a connection or group naming a layer that has no such name.

The suggestion searches the options that type actually accepts, so `widths` on a `pool` is caught as readily as `hieght` on a `conv`. The first is spelled correctly and still means nothing there.

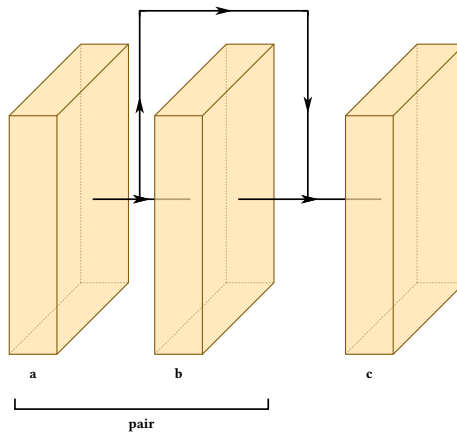
2.2 Naming a layer

name

default none

An identifier for this layer. Never printed. Connections and groups find a layer by it, and nothing else does, so a layer with no name cannot be referred to.

```
#draw-network(
(
  conv(name: "a", label: "a"),
  conv(name: "b", label: "b"),
  conv(name: "c", label: "c"),
),
// names are how connections and groups find a layer
connections: (connection(from: "a", to: "c")),
groups: (group(from: "a", to: "b", label: "pair")),
)
```

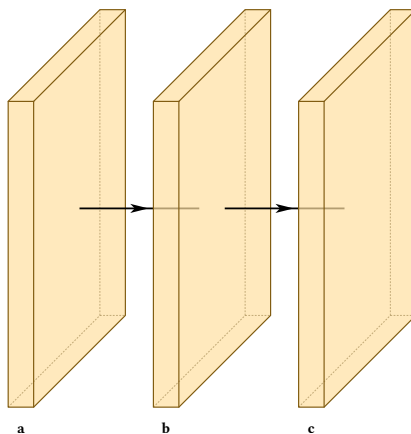


2.3 The list is just an array

Nothing requires you to write layers out one by one. The list is an array and a constructor is a function, so ordinary Typst builds both. Sharing options across several layers is a spread:

```
// Options shared by many layers, spread into each
#let big = (height: 6, depth: 6, widths: (0.5,))

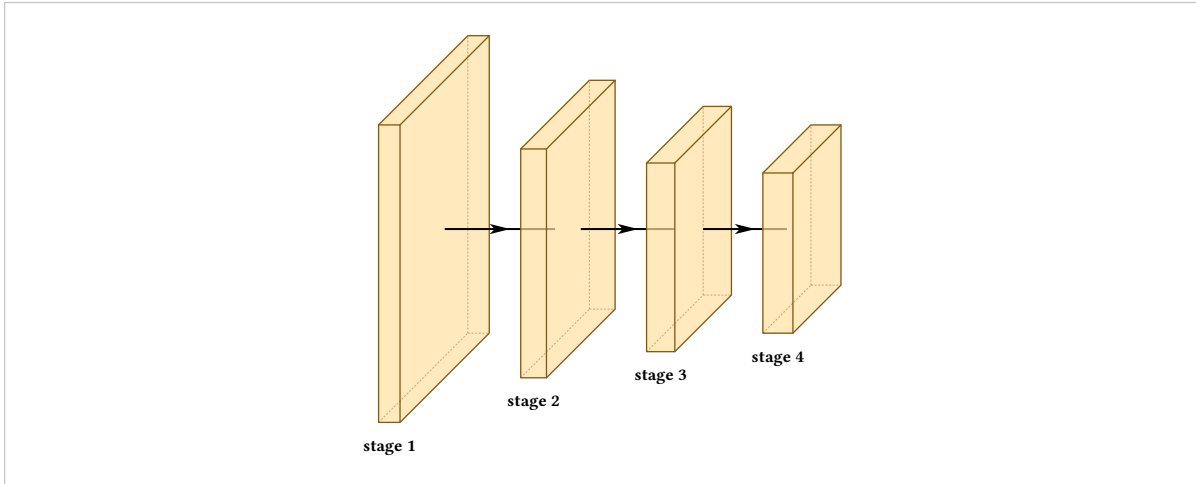
#draw-network((
  conv(label: "a", ..big),
  conv(label: "b", ..big),
  conv(label: "c", ..big),
))
```



And generating layers is a map:

```
// A layer list is an array, so build it however you like
#let stage(n) = conv(
  label: "stage " + str(n),
  shape: (32 * n, 128 / n, 128 / n),
)

#draw-network(range(1, 5).map(stage))
```

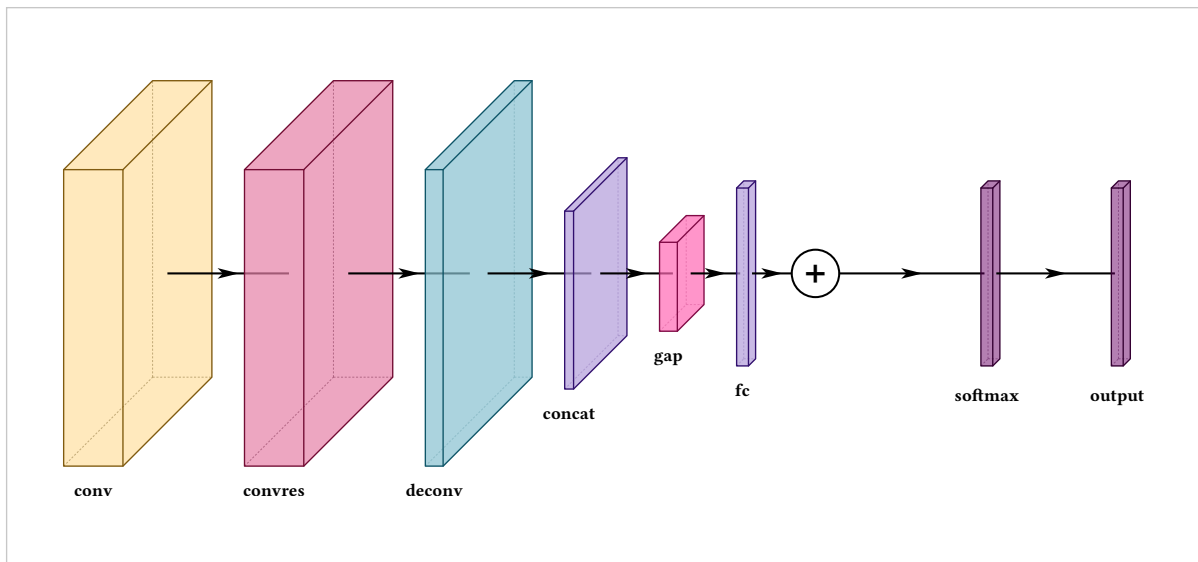


This is the difference between a figure you maintain and a figure you rewrite. Section 15.5 takes it further.

Layer types

Fourteen types are predefined. Each carries a colour, a default size, a default legend entry, and in three cases a behaviour.

```
#draw-network((
  conv(label: "conv"),
  convres(label: "convres"),
  deconv(label: "deconv"),
  concat(label: "concat"),
  gap(label: "gap"),
  fc(label: "fc"),
  sum(),
  softmax(label: "softmax"),
  output(label: "output", offset: 2),
))
```



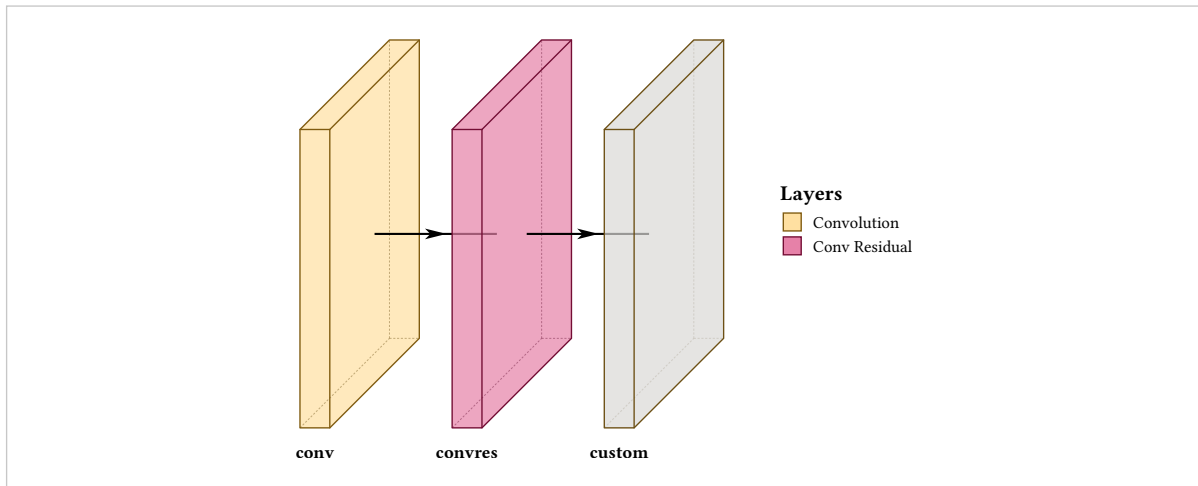
Also predefined: *input, pool, unpool, convsoftmax, custom*.

Type governs colour and defaults, not capability. Any type can be resized, recoloured, relabelled and given an image. Choose by what the block **means**: the type decides what the legend calls it and what colour a reader will come to associate with it across your figures.

3.1 The convolution family

`conv` and `convres` differ only in colour and legend text. `custom` is the same block with no meaning attached, and is the one you extend (chapter 9).


```
#draw-network(
(
  conv(label: "conv", widths: (0.5,)),
  convres(label: "convres", widths: (0.5,)),
  custom(label: "custom", width: 0.5),
),
show-legend: true,
)
```



3.2 Pooling attaches to the block in front

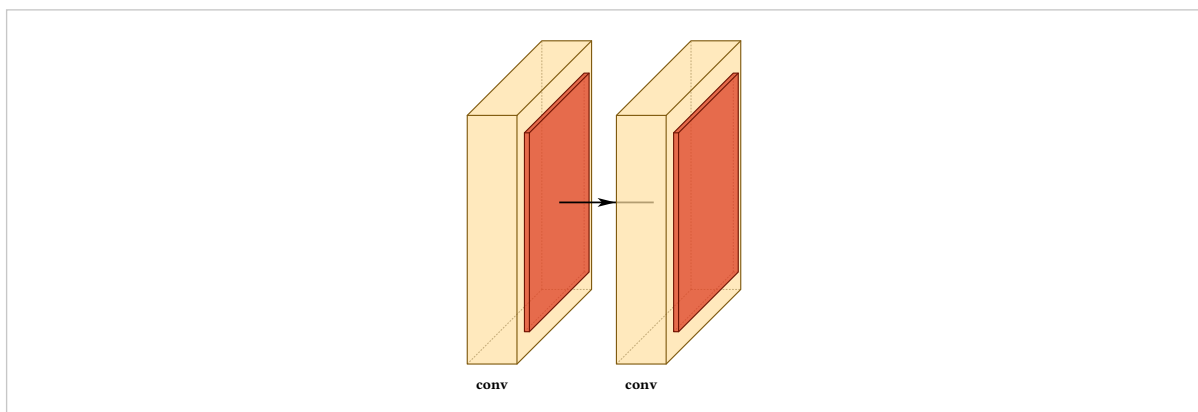
offset

default auto

read specially by pool and unpool

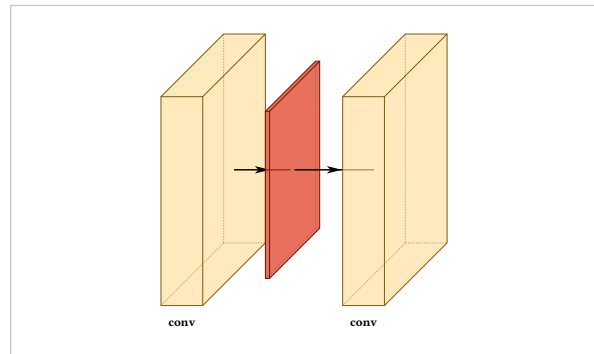
On pool and unpool, leaving offset unset attaches the layer to the block before it, with no arrow between them. That is what makes a convolution-plus-pool read as one stage rather than two.

```
// No offset: the pool sits against the block in front
#draw-network((
  conv(label: "conv"),
  pool(),
  conv(label: "conv"),
  pool(),
))
```



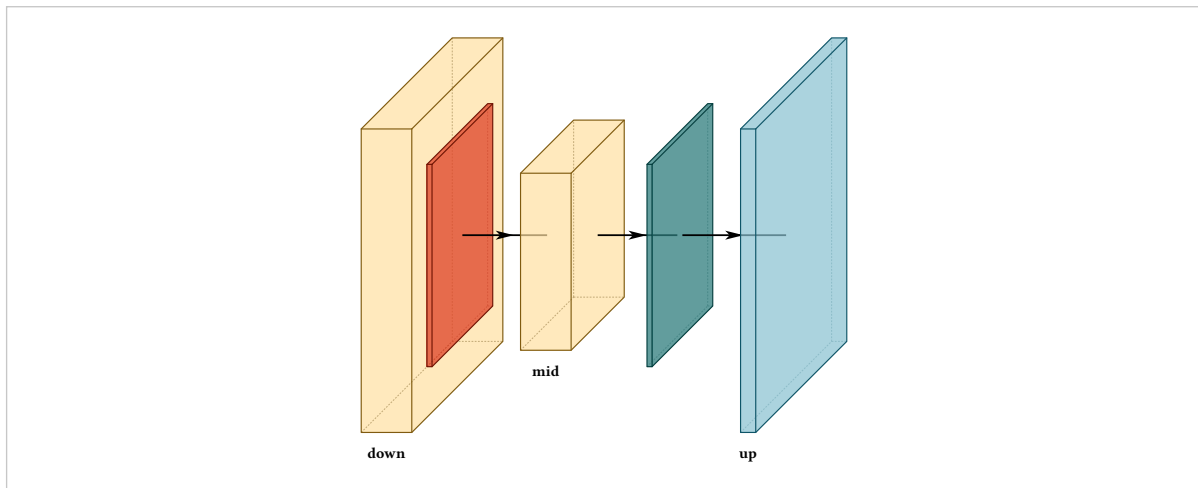
Give one an offset and it detaches, and the axis arrow appears:

```
// An offset detaches it and draws an arrow
#draw-network((
  conv(label: "conv"),
  pool(offset: 1.5),
  conv(label: "conv"),
))
```



An encoder-decoder uses both, pool on the way down and unpool on the way up:

```
#draw-network((
  conv(label: "down", height: 6, depth: 6),
  pool(height: 4, depth: 4),
  conv(label: "mid", height: 3.5, depth: 3.5),
  unpool(height: 4, depth: 4, offset: 1.5),
  deconv(label: "up", height: 6, depth: 6),
))
```



3.3 The sum node

sum is a node, not a block. It has a radius where other types have a width, and a symbol you can replace.

radius

default 0.4

sum

Size of the disc.

symbol

default +

sum

Any content, centred in the disc.

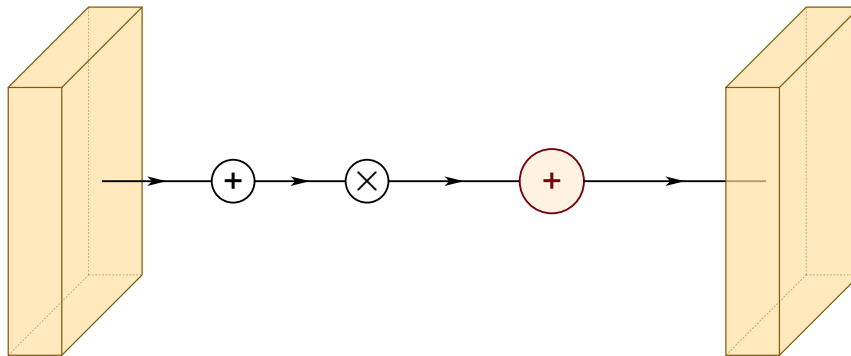
stroke

default black

sum

Outline colour.

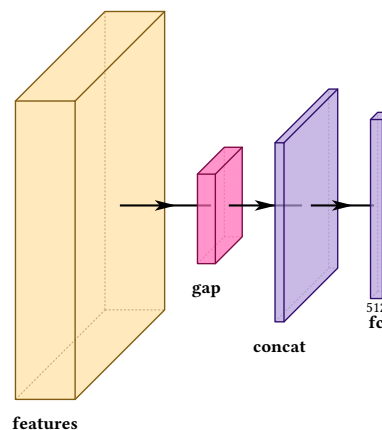
```
#draw-network((
  conv(),
  sum(),
  sum(symbol: [$times$]),
  sum(radius: 0.6, fill: rgb("#FF2E3"), stroke: rgb("#73000A")),
  conv(),
))
```



3.4 Pooling to a vector, and joining

gap collapses a feature map to a vector and is small by default. concat joins two paths, and is the usual target for a fan of arriving routes (section 10.6).

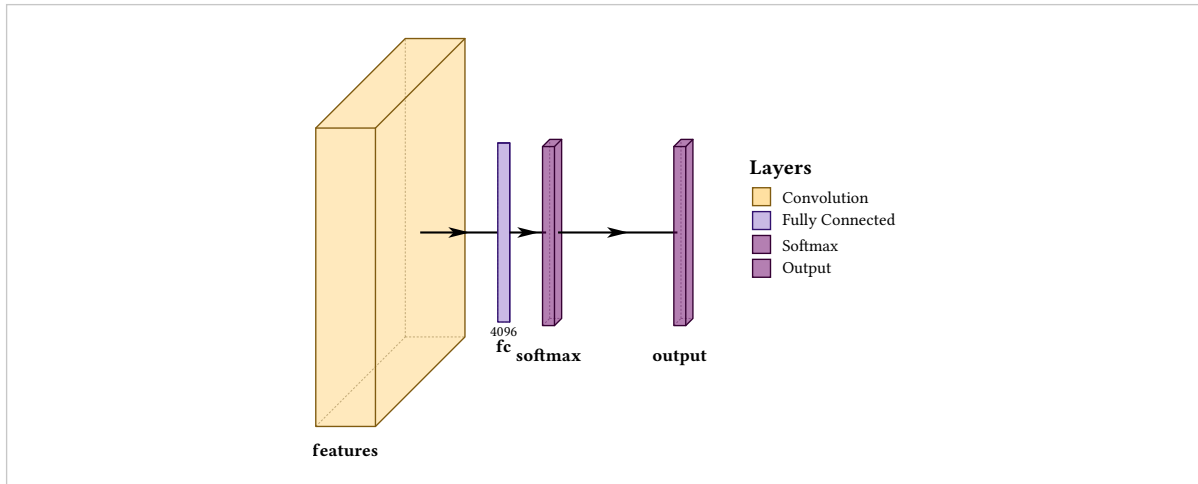
```
#draw-network((
  conv(label: "features", height: 5, depth: 5),
  gap(label: "gap"),
  concat(label: "concat"),
  fc(label: "fc", channels: (512,)),
))
```



3.5 Classifier heads

fc, softmax, convsoftmax and output are the tail of a classifier. fc with depth: 0 draws as a flat rectangle, which is what a vector looks like.

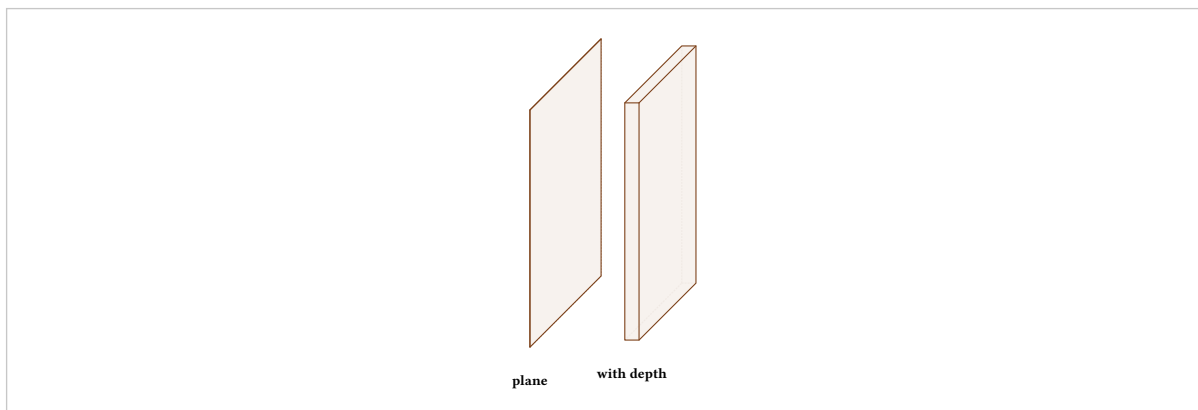
```
#draw-network(
(
  conv(label: "features"),
  fc(label: "fc", channels: (4096,), depth: 0),
  softmax(label: "softmax"),
  output(label: "output", offset: 2),
),
show-legend: true,
)
```



3.6 The input layer

input defaults to a flat plane with no thickness, which is what an image is. Give it a width and a depth to make it a block instead. It is also the one type with no outgoing arrow by default (section 10.9).

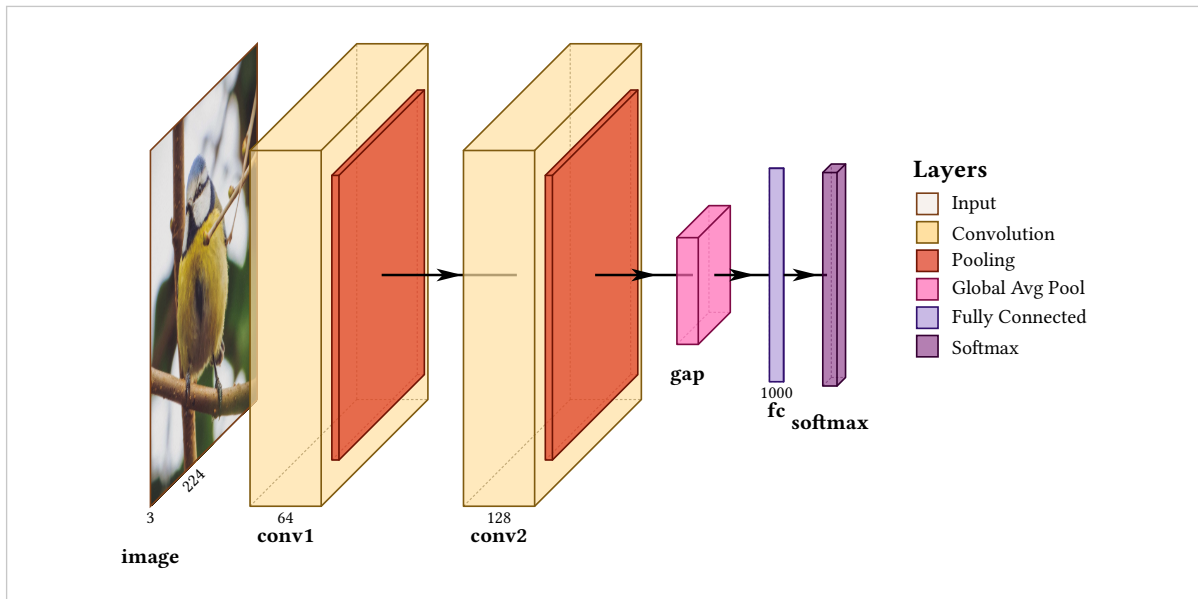
```
#draw-network((
  input(label: "plane"),
  input(label: "with depth", depth: 4, width: 0.3, offset: 2),
))
```



3.7 Putting the types together

Nothing below sets a size, a colour or a gap. It is types, labels and channel counts only, and it is already a readable figure.

```
// A whole small classifier, using nothing but types and labels
#draw-network(
  (
    input(image: "default", label: "image", channels: (3, 224)),
    conv(label: "conv1", channels: (64,), offset: 1.4),
    pool(),
    conv(label: "conv2", channels: (128,)),
    pool(),
    gap(label: "gap"),
    fc(label: "fc", channels: (1000,), depth: 0),
    softmax(label: "softmax"),
  ),
  show-legend: true,
)
```



Sizing a block

A block has three dimensions. height and depth are the spatial extent of the tensor; thickness is width or widths, and conventionally means channel count.

height

default 5, varies by type

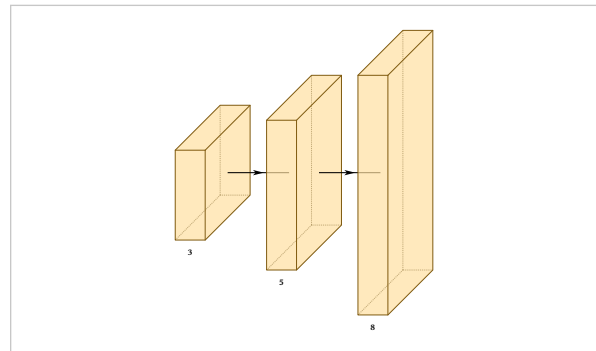
Vertical extent, in canvas units.

depth

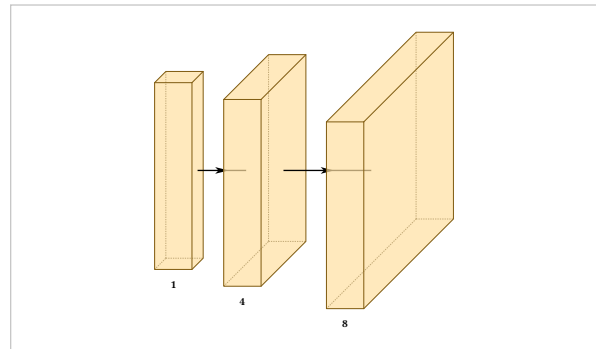
default 5, varies by type

Extent along the projection. 0 draws a flat rectangle.

```
#draw-network((
  conv(label: "3", height: 3),
  conv(label: "5", height: 5),
  conv(label: "8", height: 8),
))
```



```
#draw-network((
  conv(label: "1", depth: 1),
  conv(label: "4", depth: 4),
  conv(label: "8", depth: 8),
))
```



4.1 Thickness

width

default varies

input, deconv, concat, convsoftmax, softmax, output, custom

A single thickness.

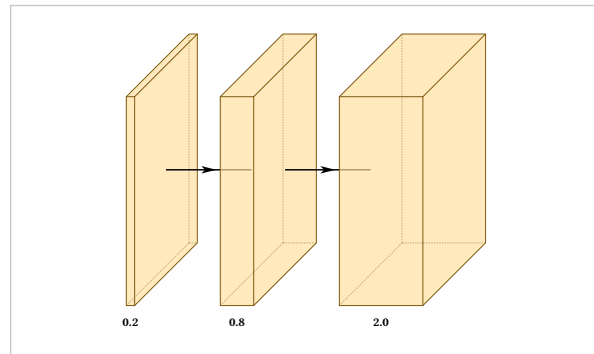
widths

default (1,)

conv, convres, custom

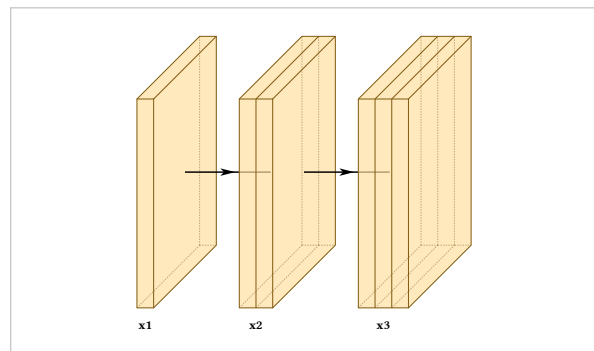
An array of thicknesses, drawing one band per entry inside a single block.

```
#draw-network((
  conv(label: "0.2", widths: (0.2,)),
  conv(label: "0.8", widths: (0.8,)),
  conv(label: "2.0", widths: (2.0,)),
))
```



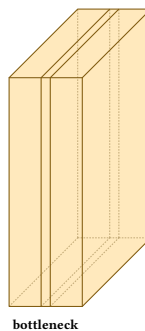
widths is how a stage of two or three convolutions at one resolution is normally drawn: one block, banded, rather than three blocks in a row.

```
#draw-network((
  conv(label: "x1", widths: (0.4,)),
  conv(label: "x2", widths: (0.4, 0.4)),
  conv(label: "x3", widths: (0.4, 0.4, 0.4)),
))
```



The bands need not be equal, which is how you draw a bottleneck:

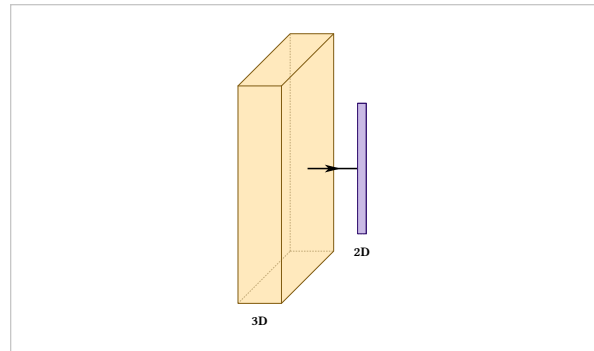
```
// Bands need not be equal
#draw-network((
  conv(label: "bottleneck", widths: (0.7, 0.2, 0.7)),
))
```



width and widths are different options and not interchangeable. No type accepts both except custom, and using the wrong one is an error naming the other.

4.2 Flat layers

```
#draw-network((  
  conv(label: "3D", depth: 4),  
  fc(label: "2D", depth: 0),  
))
```



4.3 Sizing from the tensor shape

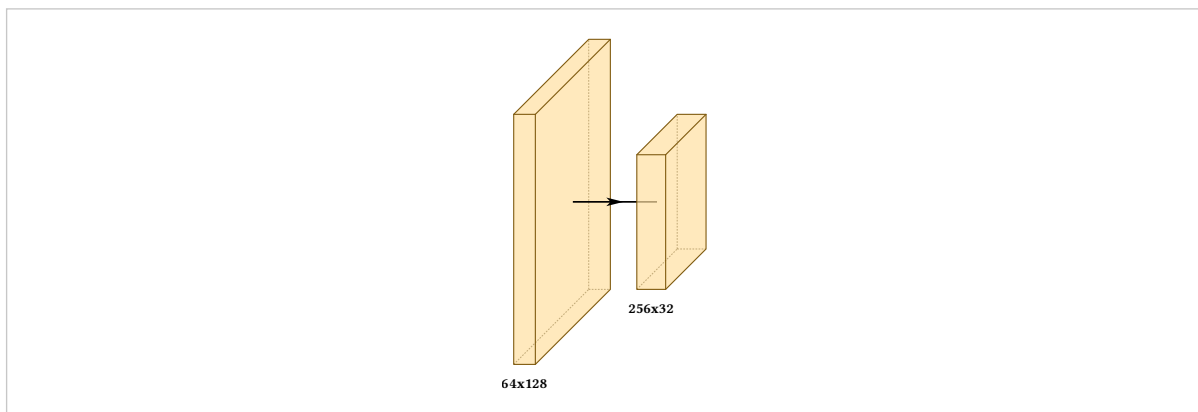
Stating three numbers per block and keeping them consistent across a pyramid is the tedious part of drawing one of these, and the part that rots when the model changes.

shape

default none

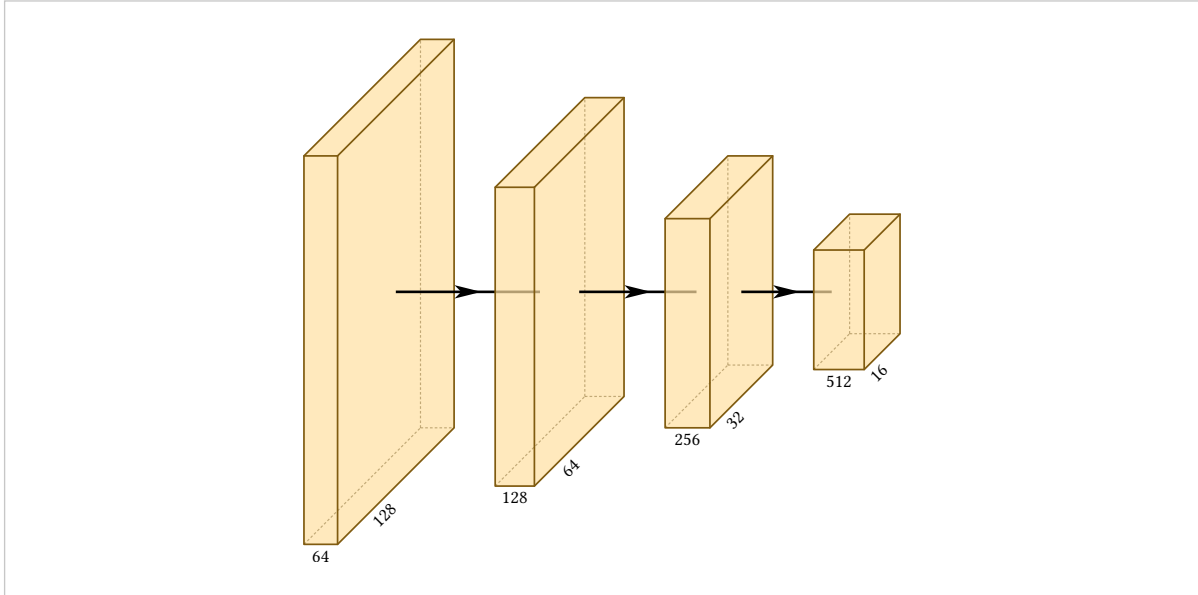
The tensor this layer produces, as (channels, height, width). Height, depth and width are derived from it.

```
// (channels, height, width) of the tensor produced  
#draw-network((  
  conv(shape: (64, 128, 128), label: "64x128"),  
  conv(shape: (256, 32, 32), label: "256x32"),  
))
```



Across a pyramid it is the whole geometry:


```
#draw-network((
  conv(shape: (64, 128, 128), channels: (64, 128)),
  conv(shape: (128, 64, 64), channels: (128, 64)),
  conv(shape: (256, 32, 32), channels: (256, 32)),
  conv(shape: (512, 16, 16), channels: (512, 16)),
))
```



Both axes are logarithmic:

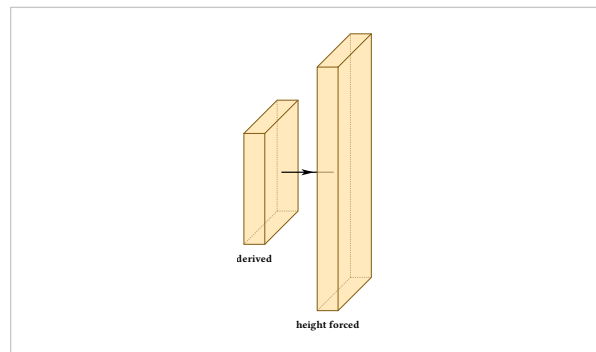
$$\text{height, depth} = 1.2 \log_2(\text{spatial}) - 3.2, \quad \text{width} = 0.075 \log_2(\text{channels})$$

Linear spatial extent does not work. Across a 640-to-20 pyramid it puts the smallest block at a quarter of a unit against 8 for the input, which is invisible.

Shape supplies defaults, not values

Anything you state explicitly wins, field by field, so a block can take its width and depth from its shape while its height is forced:

```
#draw-network((
  conv(label: "derived", shape: (256, 40, 40)),
  conv(
    label: "height forced",
    shape: (256, 40, 40),
    height: 7,
  ),
))
```



Changing the mapping

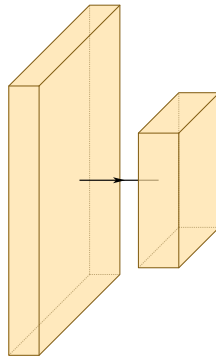
shape-scale

default (spatial: (1.2, -3.2), channels: (0.075, 0.0))

draw-network

Slope and intercept for each axis, applied to the base-2 logarithm. The constants are absolute rather than normalised across a figure, so the same shape gives the same size everywhere and two figures stay comparable.

```
// Steeper: same shapes, more separation between them
#draw-network(
  (conv(shape: (64, 128, 128)), conv(shape: (256, 32, 32))),
  shape-scale: (
    spatial: (2.0, -6.0),
    channels: (0.15, 0.0),
  ),
)
```

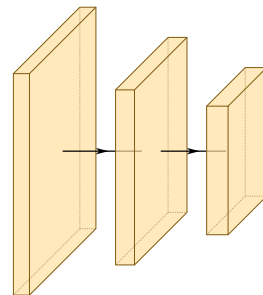


Only conv, convres and custom take a derived width, because only those three use thickness to mean channel count. The others have a fixed thickness that says something else, and keep it.

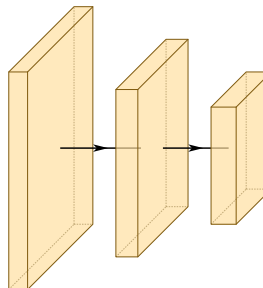
4.4 By hand against by shape

The same three blocks, written both ways. The first drifts the moment the model changes; the second cannot.

```
// The same three blocks, sized by eye
#draw-network((
  conv(height: 6.2, depth: 6.2, widths: (0.45,)),
  conv(height: 4.9, depth: 4.9, widths: (0.53,)),
  conv(height: 3.6, depth: 3.6, widths: (0.60,)),
))
```



```
// ...and sized from what they actually are
#draw-network((
  conv(shape: (64, 128, 128)),
  conv(shape: (128, 64, 64)),
  conv(shape: (256, 32, 32)),
))
```



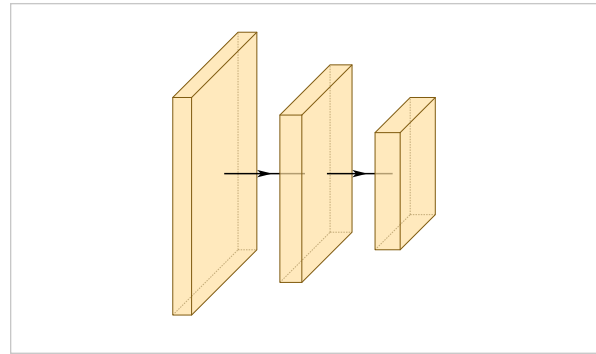
Spacing

offset

default auto

The gap between this layer and the one before it. `auto` computes it from the drawing; a number states it directly.

```
// No offsets stated anywhere
#draw-network((
  conv(shape: (64, 128, 128)),
  conv(shape: (128, 64, 64)),
  conv(shape: (256, 32, 32)),
))
```

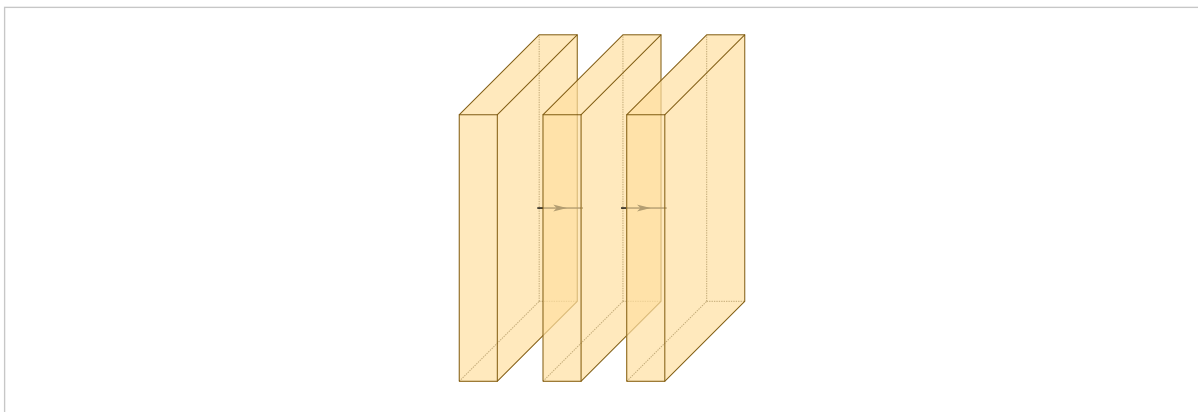


Automatic spacing covers the previous block's isometric lean, adds a fixed strip of white space, and widens where a connection descends into the gap.

5.1 Why a constant offset fails

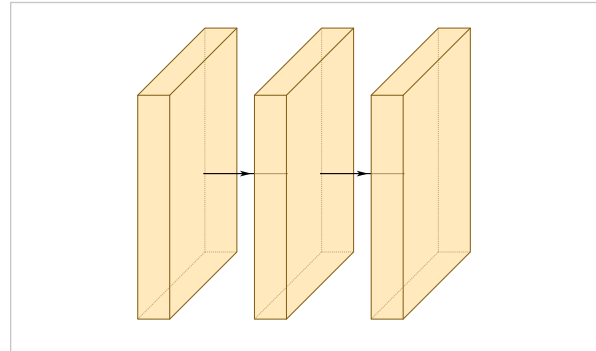
A block drawn in isometric projection leans right by $\text{depth} \times \text{depth-multiplier}$. A gap narrower than that lean buys no visible space at all: the next block is drawn inside the previous one's sheared face, and the arrow between them disappears behind it.

```
// A fixed offset narrower than the block's own lean
#draw-network((
  conv(height: 7, depth: 7),
  conv(height: 7, depth: 7, offset: 1.2),
  conv(height: 7, depth: 7, offset: 1.2),
))
```



offset: 1.2 against a depth-7 block, whose lean is 2.1.

```
// The same blocks, spaced automatically
#draw-network((
  conv(height: 7, depth: 7),
  conv(height: 7, depth: 7),
  conv(height: 7, depth: 7),
))
```



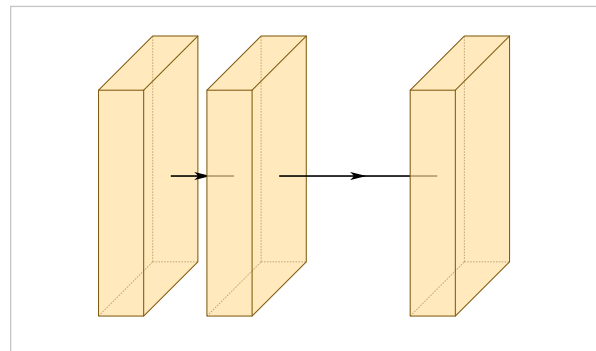
The same blocks with the offsets removed.

That is why `auto` is the default. A constant that works for one size of block fails for the next, and a figure that computes its own offsets ends up restating the size pyramid twice, with nothing to catch the two copies drifting apart.

5.2 Spacing by hand

A number overrides the automatic gap for one layer, and means what it always meant:

```
#draw-network((
  conv(depth: 4),
  conv(depth: 4, offset: 1.4),
  conv(depth: 4, offset: 3.5),
))
```

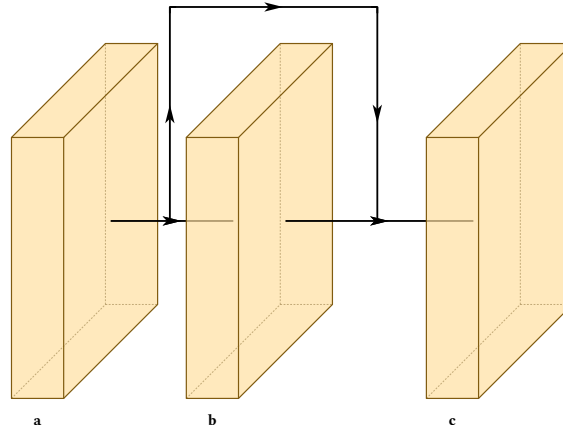


Two helpers are exported for figures that do compute their own geometry:

```
depth-shear(6)    // 1.8 -- how far a depth-6 block leans right
min-clear-offset(6) // 3.6 -- smallest gap a connection can descend into
```

A connection descending between two blocks arrives at the midpoint of the arrow joining them, so it clears the lean only when half the offset exceeds it. That is what `min-clear-offset` returns, and what automatic spacing applies for you:

```
// The gap before "c" widens on its own to let the route down
#draw-network(
  (
    conv(name: "a", label: "a", depth: 6),
    conv(name: "b", label: "b", depth: 6),
    conv(name: "c", label: "c", depth: 6),
  ),
  connections: (connection(from: "a", to: "c")),
)
```



Both helpers take a `depth-multiplier` argument, which must match the one passed to `draw-network`. The default multiplier of 0.3 has no exact binary representation, so `depth-shear(4.5)` is 1.3499999999999999. Compare with a tolerance, if you compare at all.

5.3 The white space constant

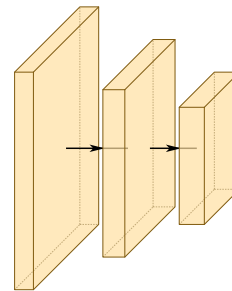
auto-gap

default 0.55

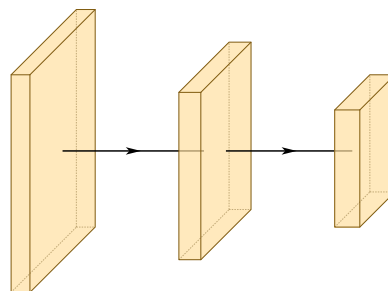
draw-network

How much visible white space an automatic offset adds, once the lean is covered.

```
#draw-network(
  (
    conv(shape: (64, 128, 128)),
    conv(shape: (128, 64, 64)),
    conv(shape: (256, 32, 32)),
  ),
  auto-gap: 0.1,
)
```



```
#draw-network(
  (
    conv(shape: (64, 128, 128)),
    conv(shape: (128, 64, 64)),
    conv(shape: (256, 32, 32)),
  ),
  auto-gap: 2.0,
)
```



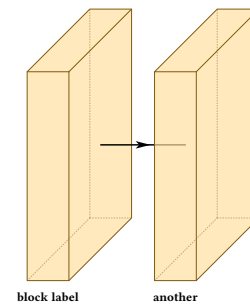
Labels

label

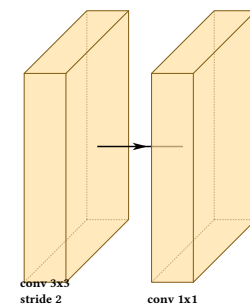
default none

Text under the block. Any content, so markup and line breaks work.

```
#draw-network((
  conv(label: "block label"),
  conv(label: "another"),
))
```



```
#draw-network((
  conv(label: [conv 3x3 \ stride 2]),
  conv(label: [conv 1x1]),
))
```



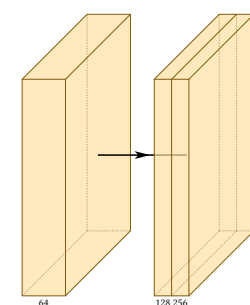
6.1 Channel numbers

channels

default none

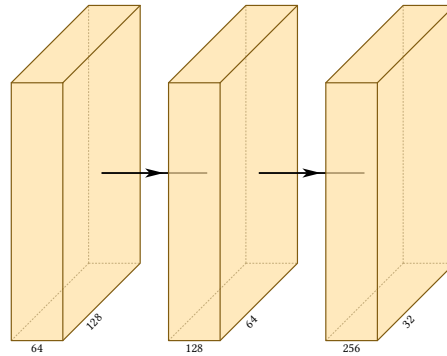
Numbers along the top of the block, one per band. At most one extra entry is allowed, and becomes the diagonal axis label. Strings work as well as numbers.

```
// one number per band
#draw-network((
  conv(channels: (64,)),
  conv(channels: (128, 256), widths: (0.4, 0.4)),
))
```



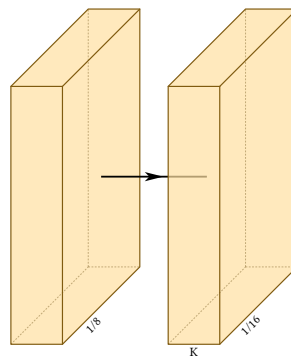
The extra entry is where spatial resolution usually goes:

```
// one more than there are bands: the extra is the axis
#draw-network((
  conv(channels: (64, 128)),
  conv(channels: (128, 64)),
  conv(channels: (256, 32)),
))
```



An empty string labels the axis and nothing else:

```
// Strings work, and an empty one labels only the axis
#draw-network((
  conv(channels: ("", "1/8")),
  conv(channels: ("K", "1/16")),
))
```



More than one extra entry is an error from inside the package, reported as an array index out of bounds rather than as a message about channels. If you see that, count your bands.

6.2 Axis names

xlabel

default none

conv, convres, custom

ylabel

default none

conv, convres, custom

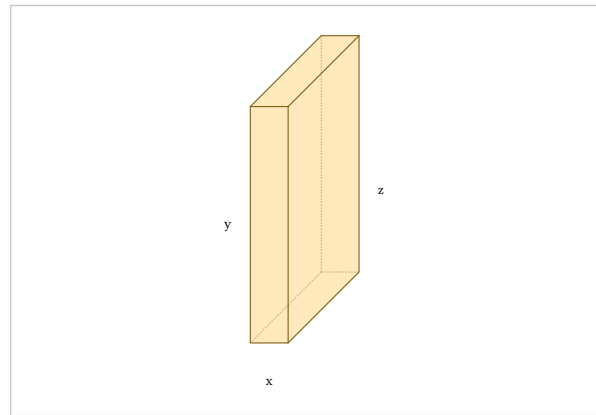
zlabel

default none

conv, convres, custom

Names for the three axes of one block, for the figure that has to explain the projection itself.

```
#draw-network((
  conv(
    xlabel: "x", ylabel: "y", zlabel: "z",
    widths: (0.8,),
  ),
))
```



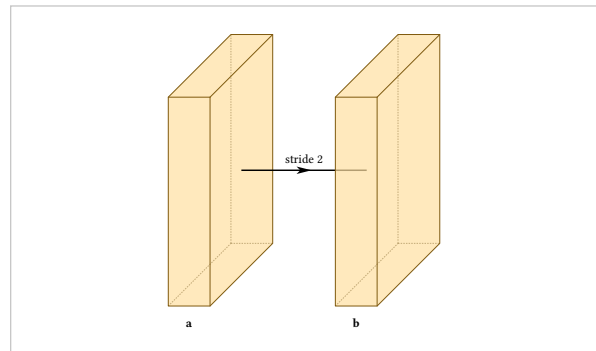
6.3 Labelling the arrow between two blocks

connection-label

default none

Text on the axis arrow **after** the layer that declares it.

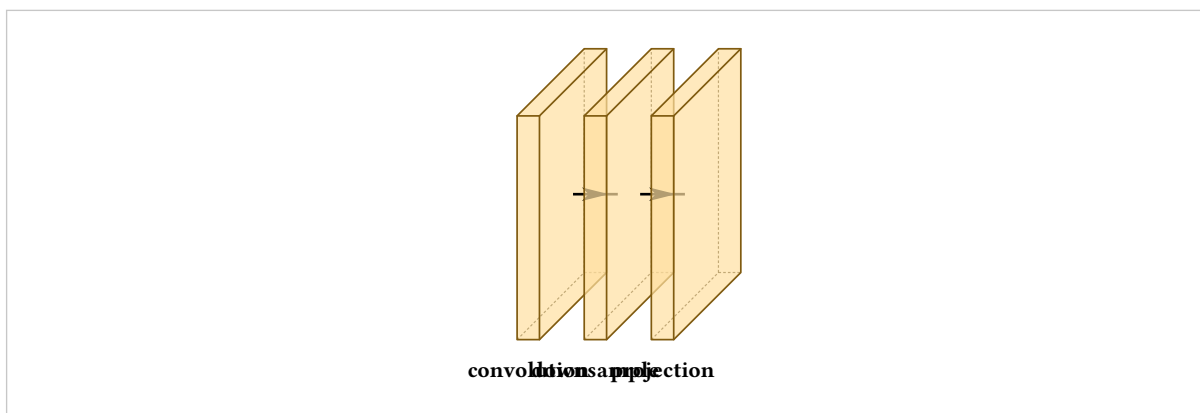
```
#draw-network((
  conv(label: "a", connection-label: "stride 2"),
  conv(label: "b", offset: 3),
))
```



6.4 When labels collide

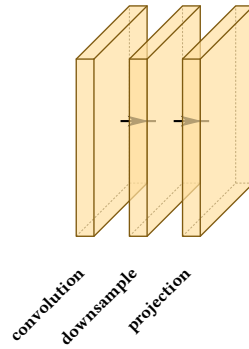
Labels sit at a fixed spot beneath each block, so long labels on closely spaced blocks overlap:

```
// Long labels, blocks close together
#let b = (height: 3, depth: 3, widths: (0.3,), offset: 0.6)
#draw-network((
  conv(label: "convolution", ..b),
  conv(label: "downsample", ..b),
  conv(label: "projection", ..b),
))
```



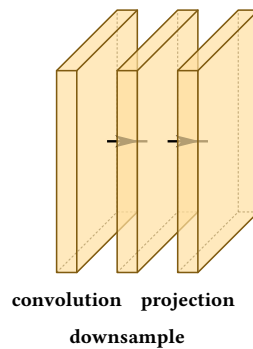
There are three fixes, and which to reach for depends on how crowded the row is. Rotating trades horizontal space, which is scarce, for vertical space, which is usually free:


```
// Rotating trades scarce horizontal space for free vertical space
#let b = (
  height: 3, depth: 3, widths: (0.3,), offset: 0.6,
  label-orient: "diagonal",
)
#draw-network((
  conv(label: "convolution", ..b),
  conv(label: "downsample", ..b),
  conv(label: "projection", ..b),
))
```



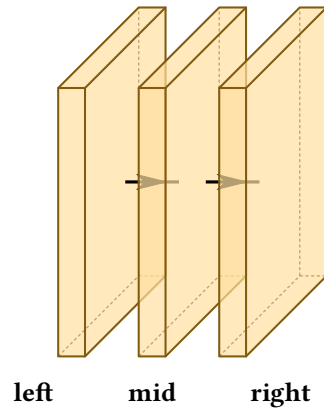
Dropping one label to a second row clears a single collision without rotating anything:

```
// Or drop the middle one to a second row
#let b = (height: 3, depth: 3, widths: (0.3,), offset: 0.6)
#draw-network((
  conv(label: "convolution", ..b),
  conv(label: "downsample", label-dy: -0.55, ..b),
  conv(label: "projection", ..b),
))
```



Anchoring the outer labels by their facing edges fans them sideways:

```
// Anchor the outer labels by their facing edges
#let b = (height: 3, depth: 3, widths: (0.3,), offset: 0.6)
#draw-network((
  conv(label: "left", label-anchor: "base-east",
    label-dx: -0.2, ..b),
  conv(label: "mid", ..b),
  conv(label: "right", label-anchor: "base-west",
    label-dx: 0.2, ..b),
))
```



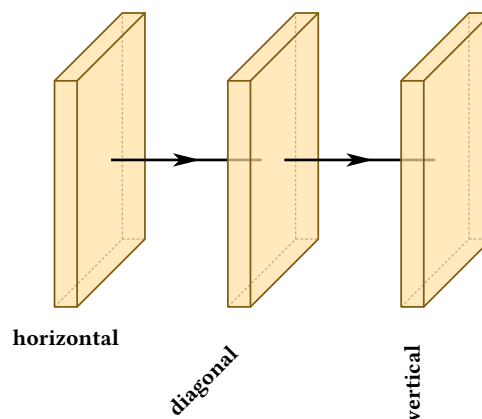
6.5 The placement options

label-orient

default "horizontal"

"horizontal", "diagonal" or "vertical". Each preset pairs its angle with the anchor that places it correctly, so a diagonal label points its end at its own block and a vertical one hangs centred beneath it. Any other value is an error.

```
#let b = (height: 3, depth: 3, widths: (0.3,), offset: 2)
#draw-network((
  conv(label: "horizontal", ..b),
  conv(label: "diagonal", label-orient: "diagonal", ..b),
  conv(label: "vertical", label-orient: "vertical", ..b),
))
```



label-dx

default 0

Horizontal shift.

label-dy

default 0

Vertical shift.

label-anchor

default set by the preset

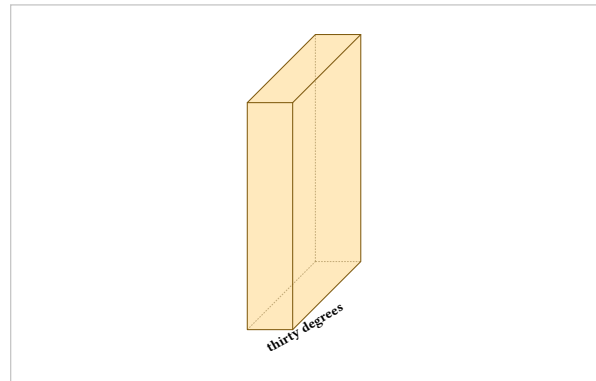
Which point of the text is pinned to the label position. Labels anchor on their baseline rather than their bounding box, so a label with descenders sits level with one without. Use "base-east" and "base-west" to anchor horizontally while keeping that alignment.

label-angle

default 0deg

An arbitrary angle. Setting both this and `label-orient` is an error, since the right anchor for an arbitrary rotation depends on where you want the text.

```
#draw-network((  
  conv(  
    label: "thirty degrees",  
    label-angle: 30deg,  
    label-anchor: "base-west",  
  ),  
))
```



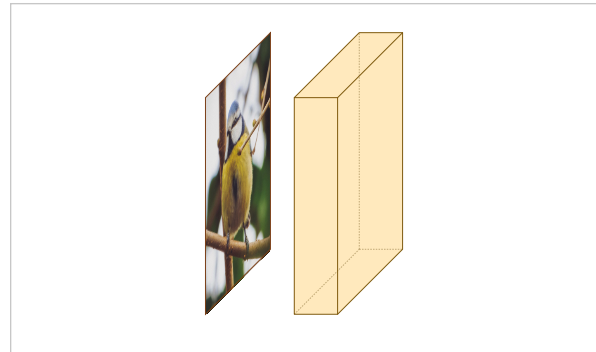
Images inside blocks

image

default none

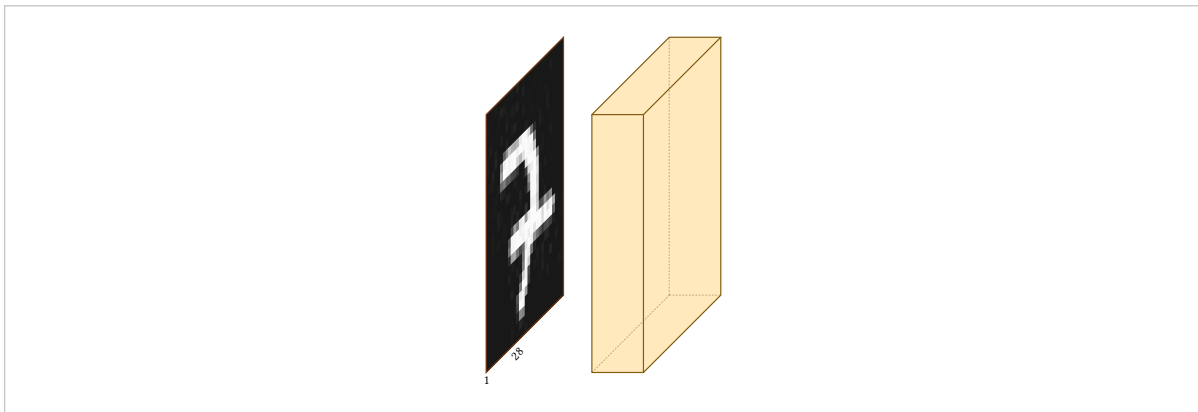
Content drawn on the block's face, sheared to match the projection. The string "default" uses the bundled photograph. Any content works, not only images.

```
#draw-network((
  input(image: "default"),
  conv(),
))
```



Any Typst image:

```
#let mnist = "../examples/networks/mnist-img-sample.jpg"
#draw-network((
  input(image: image(mnist), channels: (1, 28)),
  conv(),
))
```



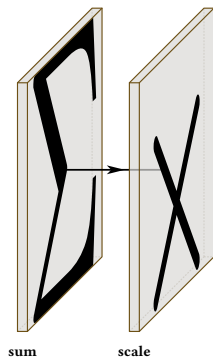
Nothing restricts this to the input. Putting a picture on a later block is how a figure shows what a stage produces:

```
// Not only the input takes one
#draw-network((
  input(image: "default", label: "in"),
  conv(image: "default", label: "features",
    depth: 4, widths: (0.4,), offset: 2),
))
```

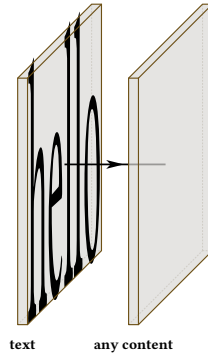


7.1 Content that is not an image

```
#draw-network((
  custom(image: [#text(20pt)[$Sigma$]], label: "sum"),
  custom(image: [#text(20pt)[$times$]], label: "scale"),
))
```



```
#draw-network((  
  custom(image: [hello], label: "text"),  
  custom(image: [#rect(width: 60%, height: 60%, fill: red)],  
    label: "any content"),  
))
```



Colour

fill

default set by the type and palette

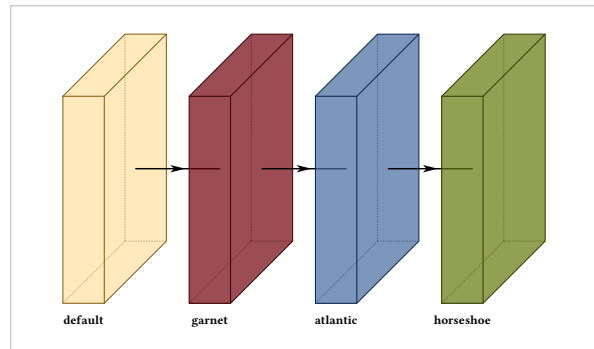
Block colour. Edge and band colours are derived from it, so a recoloured block stays internally consistent.

opacity

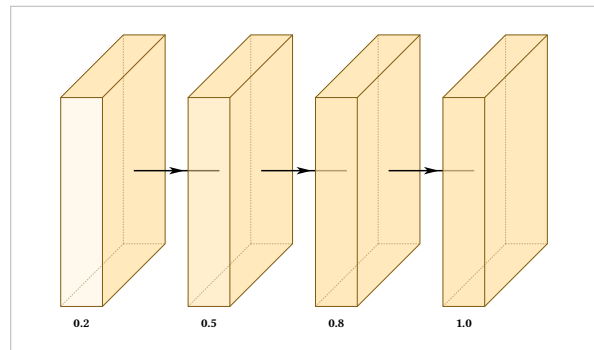
default 0.7, varies by type

How solid the block is.

```
#draw-network((
  conv(label: "default"),
  conv(label: "garnet", fill: rgb("#73000A")),
  conv(label: "atlantic", fill: rgb("#466A9F")),
  conv(label: "horseshoe", fill: rgb("#65780B")),
))
```



```
#draw-network((
  conv(label: "0.2", opacity: 0.2),
  conv(label: "0.5", opacity: 0.5),
  conv(label: "0.8", opacity: 0.8),
  conv(label: "1.0", opacity: 1.0),
))
```



8.1 Activation bands

A darker band at the end of a convolution conventionally marks the activation.

show-relu

default false

draw-network; conv, convres, custom

Turn bands on for a whole figure with the `draw-network` argument, and override on any one layer.

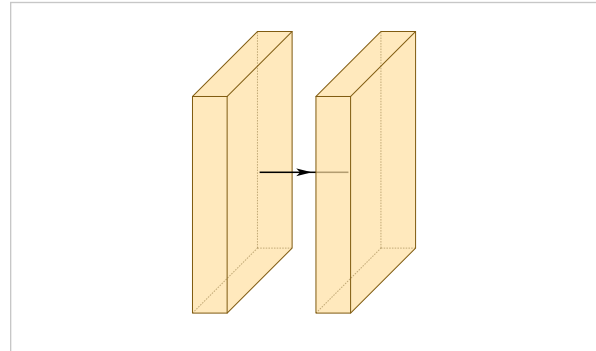
bandfill

default derived from fill

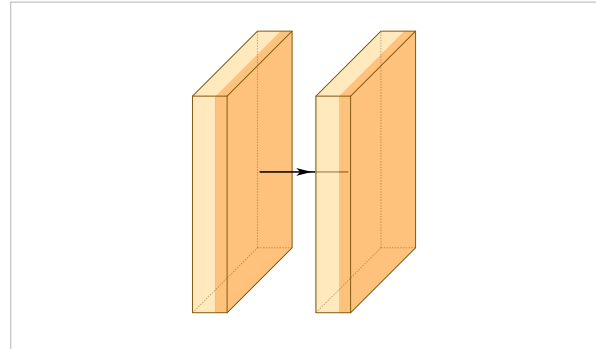
conv, convres, custom

Band colour. Undeclared, it is derived from the layer's own fill, so a recoloured block gets a matching band rather than one from the palette.

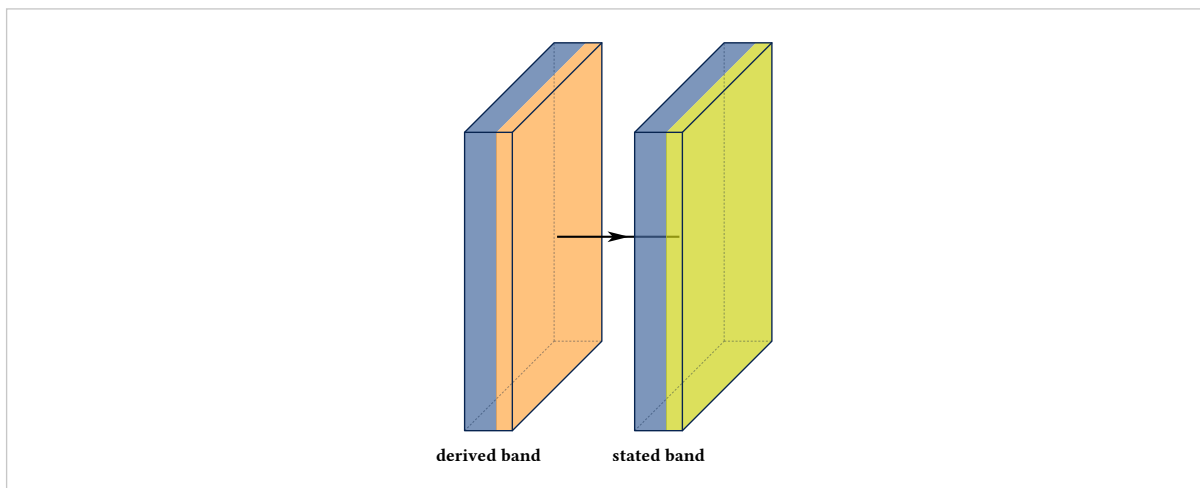
```
#draw-network(
  (conv(widths: (0.8,)), conv(widths: (0.8,))),
  show-relu: false,
)
```



```
#draw-network(
  (conv(widths: (0.8,)), conv(widths: (0.8,))),
  show-relu: true,
)
```



```
#draw-network(
  (
    conv(label: "derived band", widths: (0.8,),
      fill: rgb("#466A9F")),
    conv(label: "stated band", widths: (0.8,),
      fill: rgb("#466A9F"), bandfill: rgb("#CED318")),
  ),
  show-relu: true,
)
```



8.2 Palettes

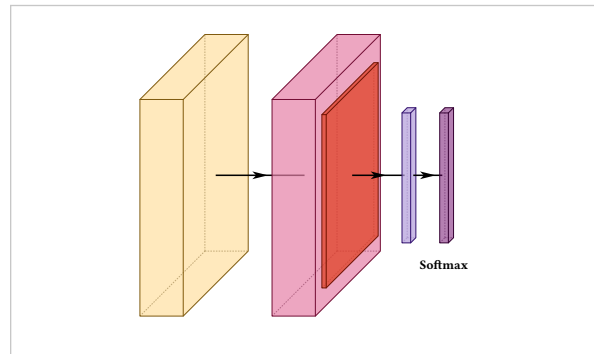
palette

default "warm"

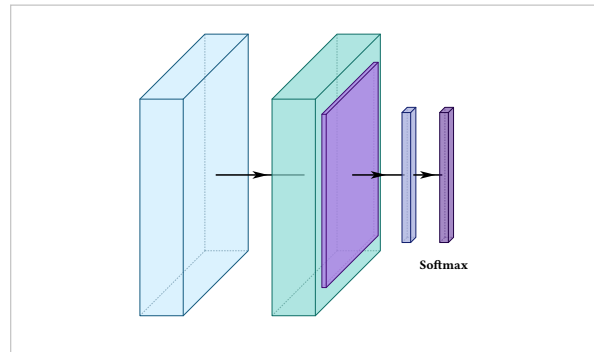
draw-network

"warm" or "cold".


```
#draw-network(
  (conv(), convres(), pool(), fc(), softmax()),
  palette: "warm",
)
```



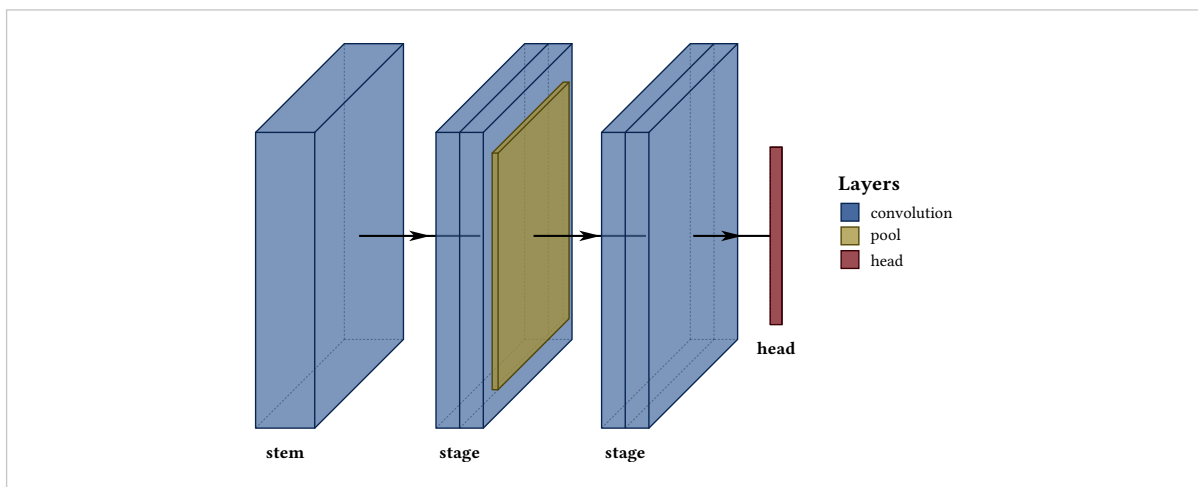
```
#draw-network(
  (conv(), convres(), pool(), fc(), softmax()),
  palette: "cold",
)
```



A palette sets defaults only. Any fill you state wins, so a figure can sit on a palette and still recolour three blocks – or ignore the palette entirely and put the whole figure on a scheme of your own:

```
// A whole figure recoloured to one scheme
#let garnet = rgb("#73000A")
#let atlantic = rgb("#466A9F")
#let honeycomb = rgb("#A49137")

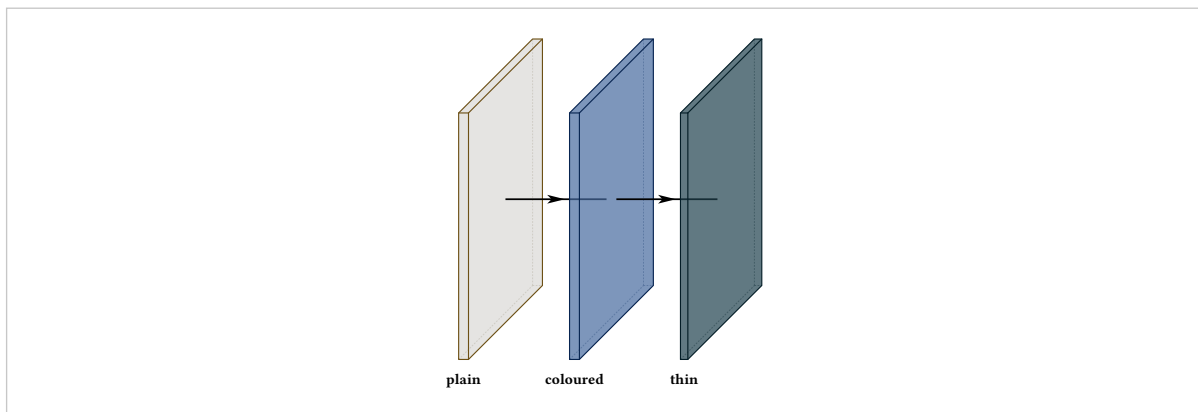
#draw-network(
  (
    conv(label: "stem", fill: atlantic, legend: "convolution"),
    conv(label: "stage", fill: atlantic, widths: (0.4, 0.4)),
    pool(fill: honeycomb, legend: "pool"),
    conv(label: "stage", fill: atlantic, widths: (0.4, 0.4)),
    fc(label: "head", fill: garnet, depth: 0, legend: "head"),
  ),
  show-legend: true,
)
```



Blocks of your own

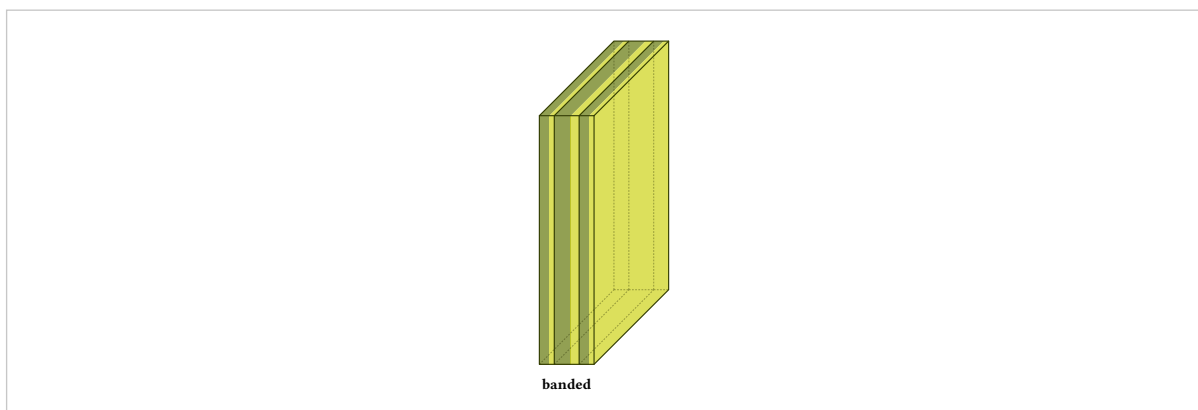
custom is the generic block: no meaning attached, every option available.

```
#draw-network((
  custom(label: "plain"),
  custom(label: "coloured", fill: rgb("#466A9F")),
  custom(label: "thin", width: 0.15, fill: rgb("#1F414D")),
))
```



It takes bands and an activation band exactly as a convolution does:

```
#draw-network(
  (
    custom(
      label: "banded",
      widths: (0.3, 0.5, 0.3),
      fill: rgb("#65780B"),
      bandfill: rgb("#CED318"),
      show-relu: true,
    ),
  ),
)
```

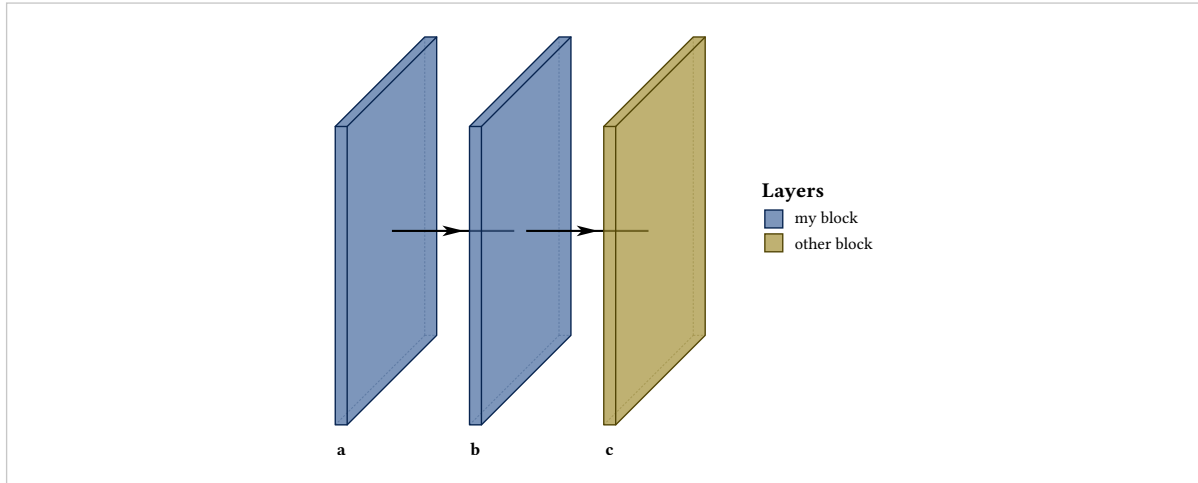


Legend

default the type's default name

What this layer is called in the legend. Layers sharing a legend name appear once, so a colour used by four blocks produces one entry.

```
#draw-network(
  (
    custom(label: "a", fill: rgb("#466A9F"), legend: "my block"),
    custom(label: "b", fill: rgb("#466A9F")),
    custom(label: "c", fill: rgb("#A49137"), legend: "other block"),
  ),
  show-legend: true,
)
```

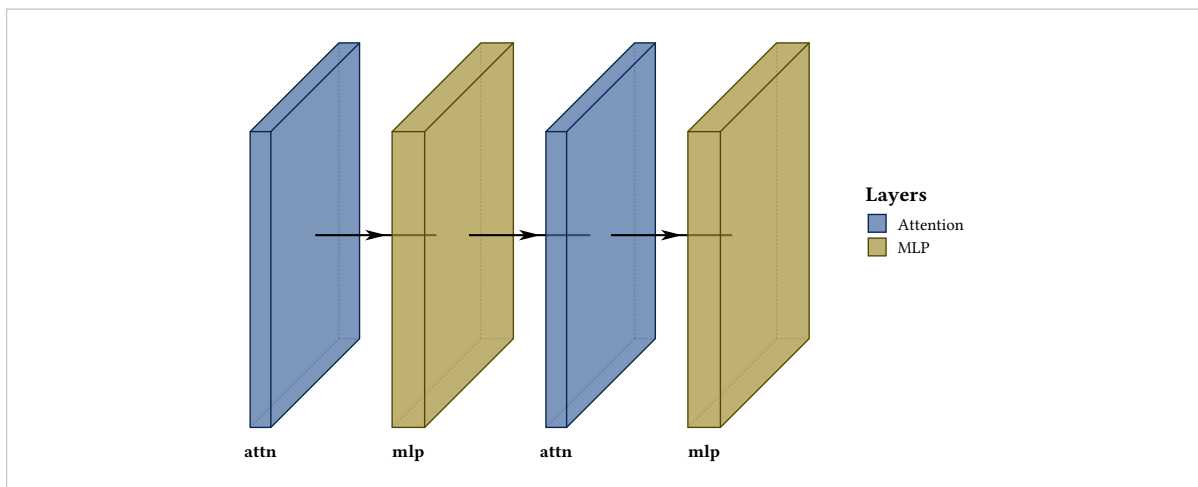


9.1 Building a vocabulary

A custom layer with its options fixed is a function returning a layer, so your own block types are a few lines. This is the intended way to draw something the package has no name for — attention, a gate, a state-space block — rather than repurposing a type that means something else to a reader.

```
// Your own block type is a function returning a layer
#let attention(..a) = custom(
  fill: rgb("#466A9F"), width: 0.35, legend: "Attention", ..a,
)
#let mlp(..a) = custom(
  fill: rgb("#A49137"), width: 0.55, legend: "MLP", ..a,
)

#draw-network(
  (attention(label: "attn"), mlp(label: "mlp"),
   attention(label: "attn"), mlp(label: "mlp")),
  show-legend: true,
)
```



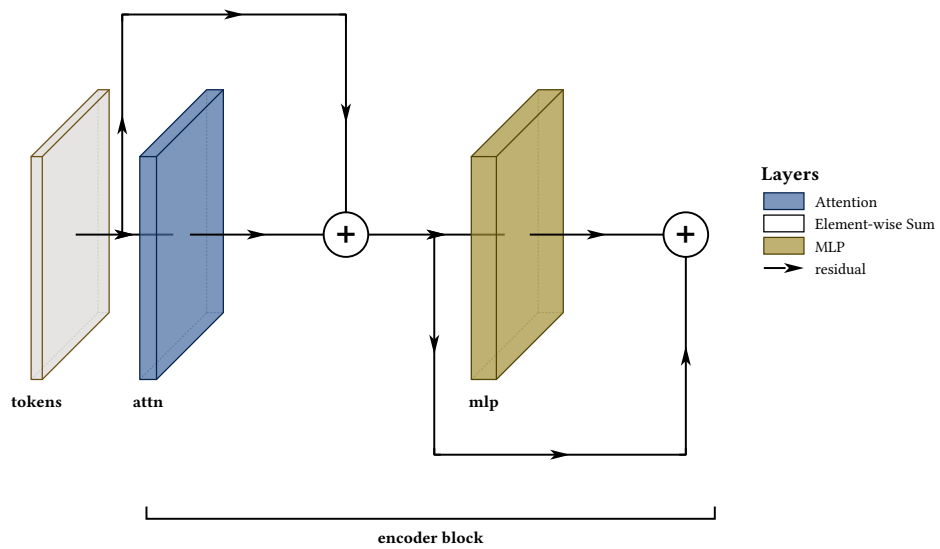
Once the vocabulary exists, a figure is written in it:

```

#let attn(..a) = custom(fill: rgb("#466A9F"), width: 0.3,
  height: 4, depth: 4, legend: "Attention", ..a)
#let mlp(..a) = custom(fill: rgb("#A49137"), width: 0.45,
  height: 4, depth: 4, legend: "MLP", ..a)

#draw-network(
  (
    custom(name: "in", label: "tokens", width: 0.2, height: 4, depth: 4),
    attn(name: "a", label: "attn"),
    sum(name: "s1"),
    mlp(name: "m", label: "mlp"),
    sum(name: "s2"),
  ),
  connections: (
    connection(from: "in", to: "s1", legend: "residual"),
    connection(from: "s1", to: "s2"),
  ),
  groups: (group(from: "a", to: "s2", label: "encoder block")),
  show-legend: true,
)

```



Connections

Arrows along the main axis are drawn for you. Anything else — a skip, a residual add, a lateral feed, an auxiliary path — goes in connections, as a list of routes between two named layers.

from

default required

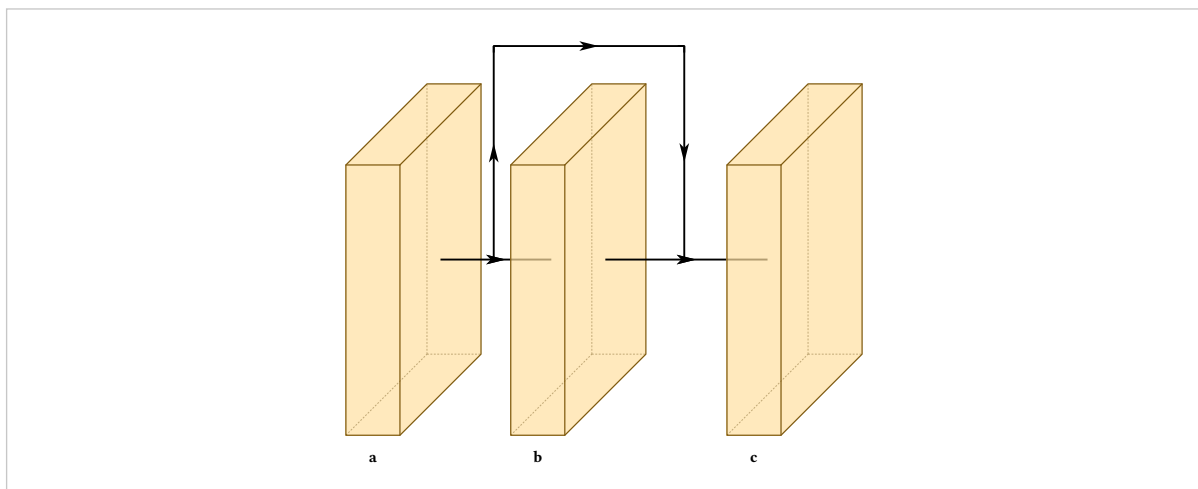
Name of the layer the route leaves.

to

default required

Name of the layer it arrives at.

```
#draw-network(
  (
    conv(name: "a", label: "a"),
    conv(name: "b", label: "b"),
    conv(name: "c", label: "c"),
  ),
  connections: (connection(from: "a", to: "c"),),
)
```



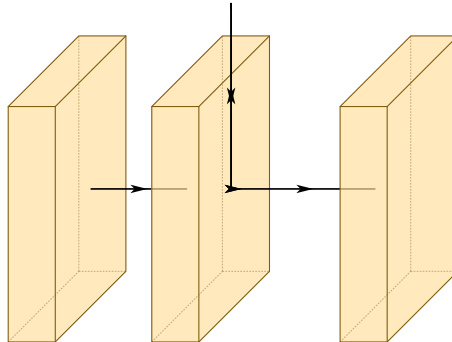
10.1 Routing modes

mode

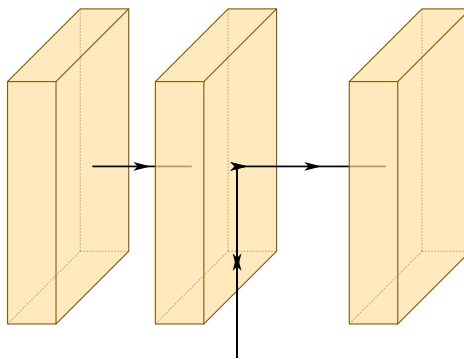
default "air"

"air" runs over the top of the stack, "flat" underneath, "depth" along the projection's own axis.

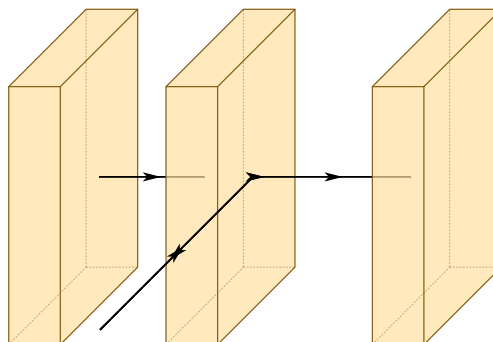
```
#draw-network(
  (conv(name: "a"), conv(), conv(name: "c")),
  connections: (connection(from: "a", to: "c", mode: "air"),),
)
```



```
#draw-network(
  (conv(name: "a"), conv(), conv(name: "c")),
  connections: (connection(from: "a", to: "c", mode: "flat"),),
)
```



```
#draw-network(
  (conv(name: "a"), conv(), conv(name: "c")),
  connections: (connection(from: "a", to: "c", mode: "depth"),),
)
```



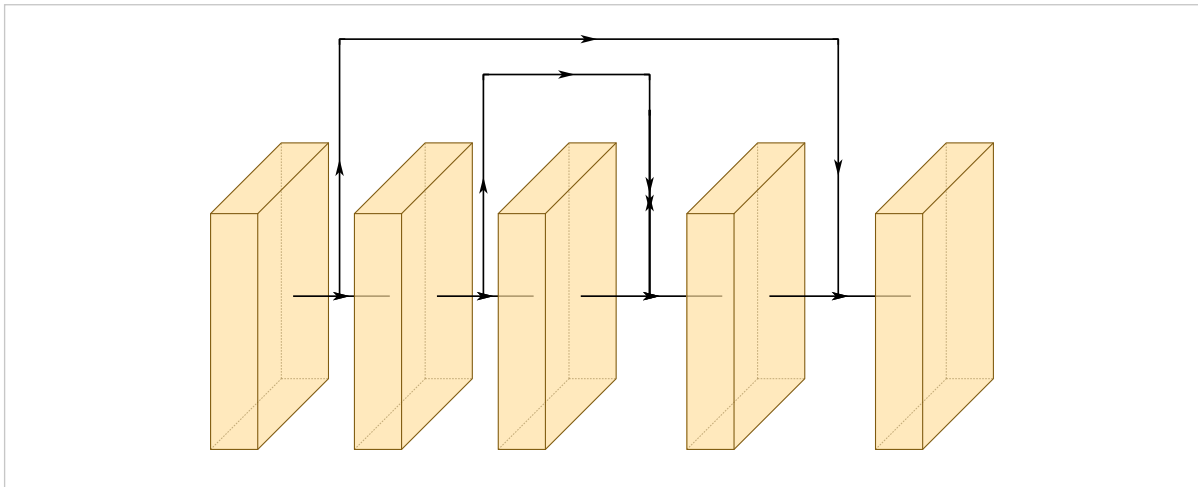
10.2 Lane heights

pos

default auto

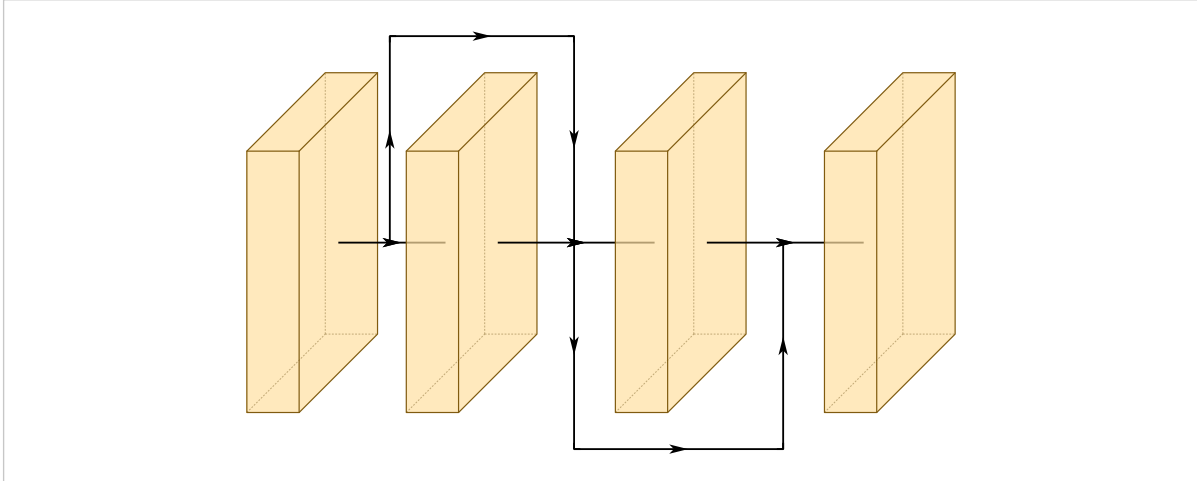
How far the route sits from the centre axis. auto ranks routes by how far they reach, so a route spanning more blocks sits higher and a longer route always arcs over a shorter one instead of crossing it.

```
// Longer routes arc over shorter ones, unasked
#draw-network(
(
  conv(name: "a"), conv(name: "b"), conv(name: "c"),
  conv(name: "d"), conv(name: "e"),
),
connections: (
  connection(from: "a", to: "e"),
  connection(from: "b", to: "d"),
  connection(from: "c", to: "d"),
),
)
```



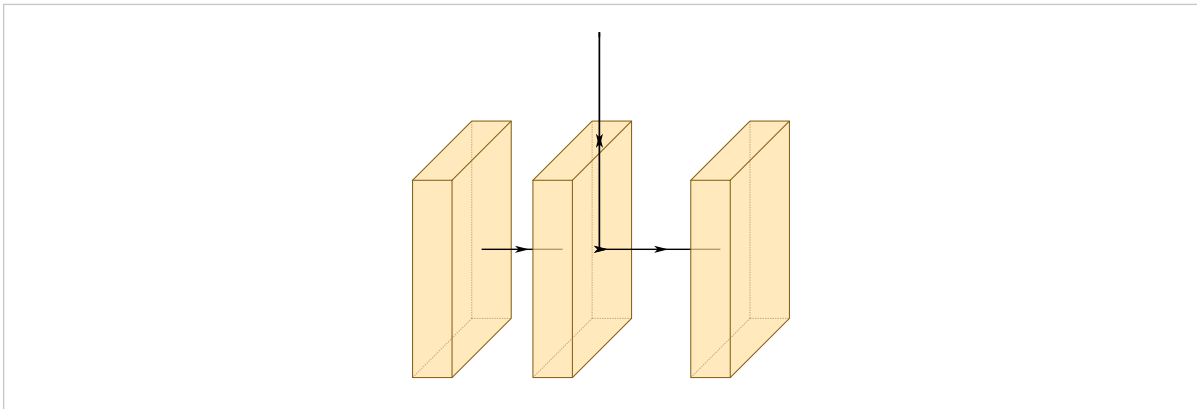
Routes of equal reach share a height. Where two of them overlap, the second goes to the opposite side of the axis at the same height, rather than being pushed further out than its reach warrants:

```
// Equal reach, overlapping: the second goes below the axis
#draw-network(
  (
    conv(name: "a"), conv(name: "b"),
    conv(name: "c"), conv(name: "d"),
  ),
  connections: (
    connection(from: "a", to: "c"),
    connection(from: "b", to: "d"),
  ),
)
```



A number overrides all of it for one route:

```
#draw-network(
  (conv(name: "a"), conv(), conv(name: "c")),
  connections: (connection(from: "a", to: "c", pos: 5.5),),
)
```



lane-unit

default 0.75

draw-network

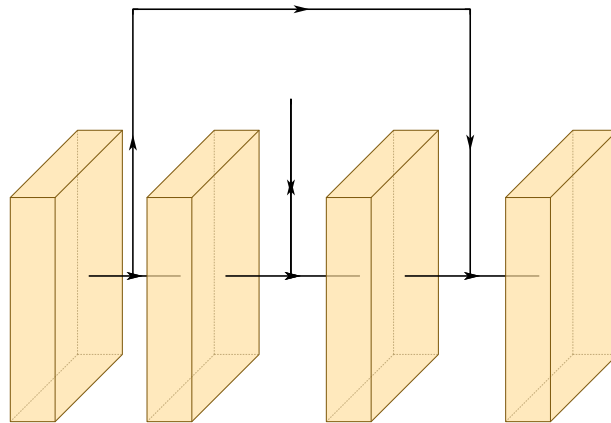
Spacing between automatic heights.

clearance

default 0.7

How far the lowest automatic route sits from the blocks.


```
#draw-network(
(
  conv(name: "a"), conv(name: "b"),
  conv(name: "c"), conv(name: "d"),
),
connections: (
  connection(from: "a", to: "d"),
  connection(from: "b", to: "c"),
),
lane-unit: 2.0,
)
```



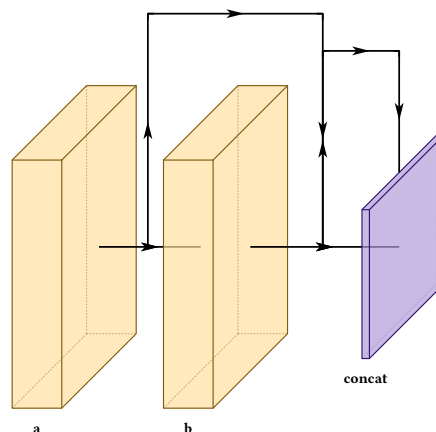
10.3 Where a route arrives

touch-layer

default false

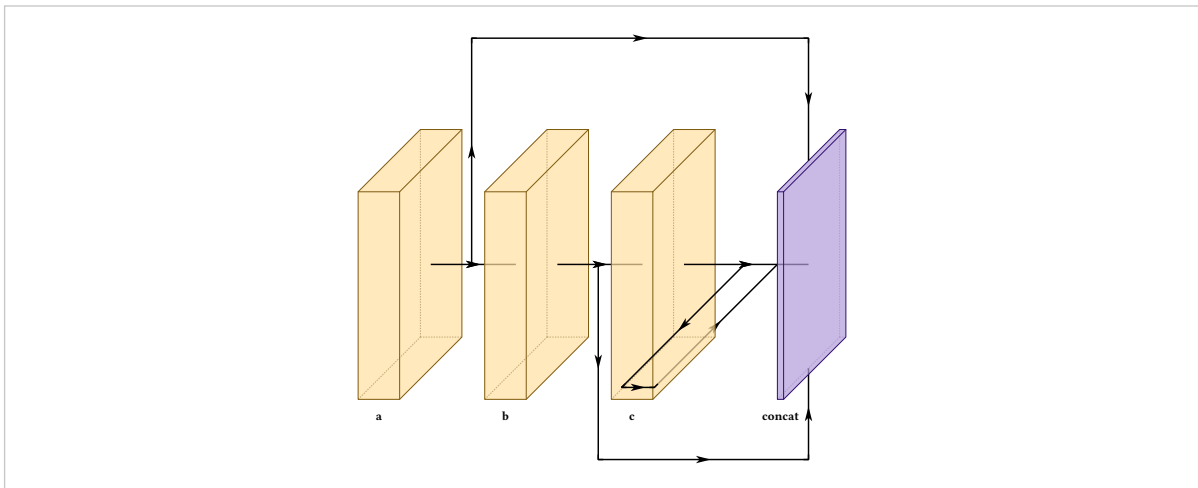
Arrive on the target block itself rather than on the axis arrow in front of it. The edge follows the routing mode: air lands on the top edge, flat on the bottom, depth on the left.

```
#draw-network(
(
  conv(name: "a", label: "a"),
  conv(name: "b", label: "b"),
  concat(name: "cat", label: "concat", depth: 5),
),
connections: (
  // on the axis arrow in front of the target
  connection(from: "a", to: "cat"),
  // on the target block itself
  connection(from: "b", to: "cat", touch-layer: true),
),
)
```



All three modes, arriving on the same block:

```
#let t = (touch-layer: true)
#draw-network(
  (
    conv(name: "a", label: "a"),
    conv(name: "b", label: "b"),
    conv(name: "c", label: "c"),
    concat(name: "cat", label: "concat", depth: 5, height: 5),
  ),
  connections: (
    connection(from: "a", to: "cat", mode: "air", ..t),
    connection(from: "b", to: "cat", mode: "flat", ..t),
    connection(from: "c", to: "cat", mode: "depth", ..t),
  ),
)
```



A route arriving on the bottom or left edge has its final stretch drawn behind the block, because those edges are on the far side and the route genuinely passes underneath before reaching them. Blocks are semi-transparent, so it shows faintly rather than disappearing.

10.4 Fanning several routes into one block

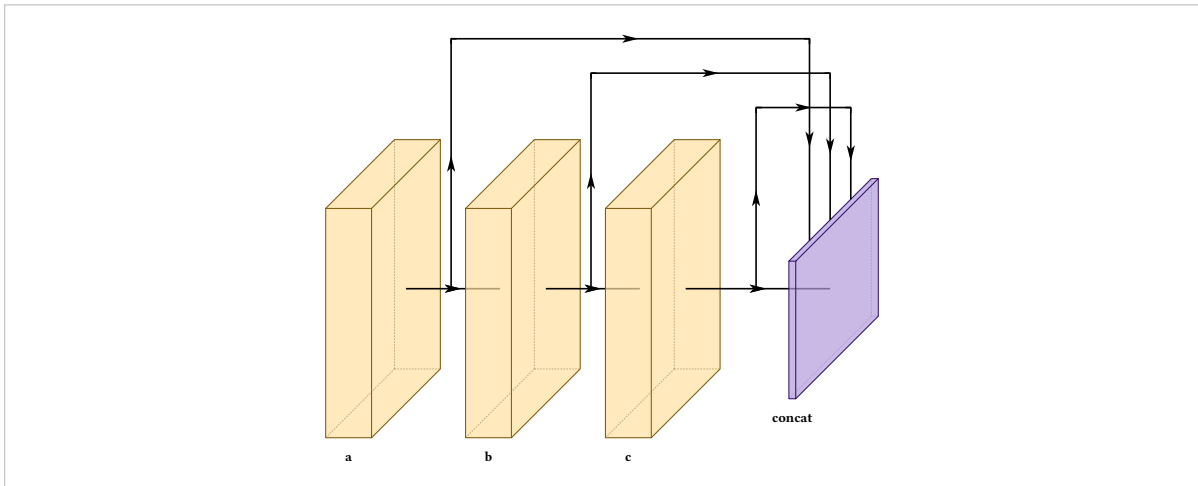
Two routes in the same mode would otherwise land on the same point, with their arrowheads stacked.

arrive-offset

default auto

Position along the arrival edge. auto spaces a whole fan evenly and leaves a lone arrival centred.

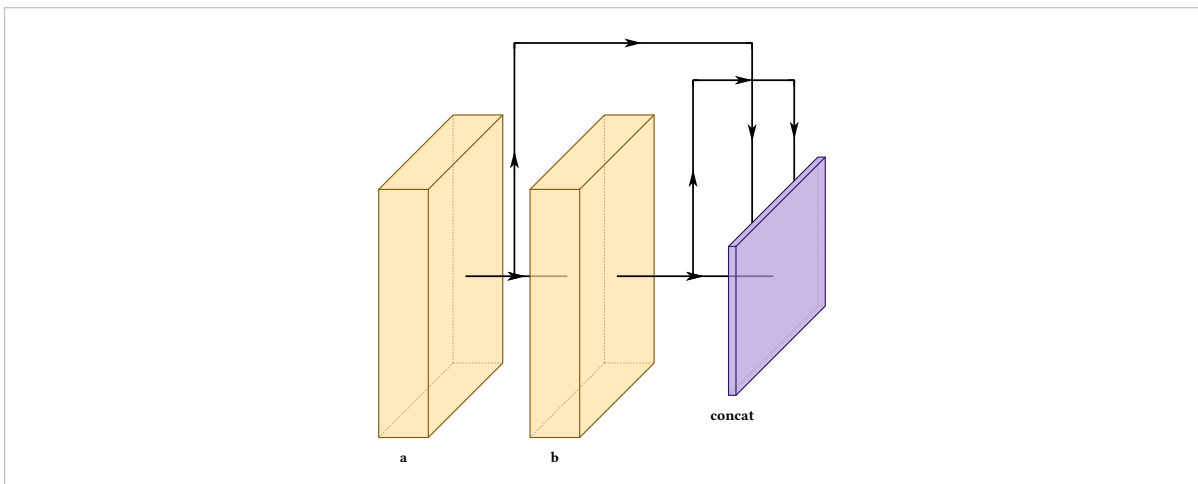
```
// Three arrivals on one edge, spaced without picking numbers
#draw-network(
(
  conv(name: "a", label: "a"), conv(name: "b", label: "b"),
  conv(name: "c", label: "c"),
  concat(name: "cat", label: "concat", depth: 6),
),
connections: (
  connection(from: "a", to: "cat", touch-layer: true),
  connection(from: "b", to: "cat", touch-layer: true),
  connection(from: "c", to: "cat", touch-layer: true),
),
)
```



Routes are grouped by the edge they land on, then spread across it: k routes divide the edge into $k + 1$ intervals and sit at the interior boundaries, so the outermost pair is inset rather than sitting on the corners. They are ordered by where each route starts, so a fan does not cross itself, and adding a route respaces the rest.

A number places one arrival yourself, measured along the edge:

```
#draw-network(
(
  conv(name: "a", label: "a"), conv(name: "b", label: "b"),
  concat(name: "cat", label: "concat", depth: 6),
),
connections: (
  connection(from: "a", to: "cat", touch-layer: true, arrive-offset: -0.6),
  connection(from: "b", to: "cat", touch-layer: true, arrive-offset: 0.6),
),
)
```



The offset runs **along the edge**, not in x . The top and bottom edges of a block's west side follow the isometric depth direction, so shifting horizontally would walk the arrival off the block. The edge is $\sqrt{2} \cdot \text{depth} \cdot \text{depth-multiplier}$ for top and bottom arrivals and the block height for left ones, so a deeper block accommodates a wider fan.

10.5 Telling routes apart

A residual add, a concat feed and an auxiliary supervision path are different things, and by default they all look the same.

color

default palette

Paint for the line and its arrowheads.

dash

default none

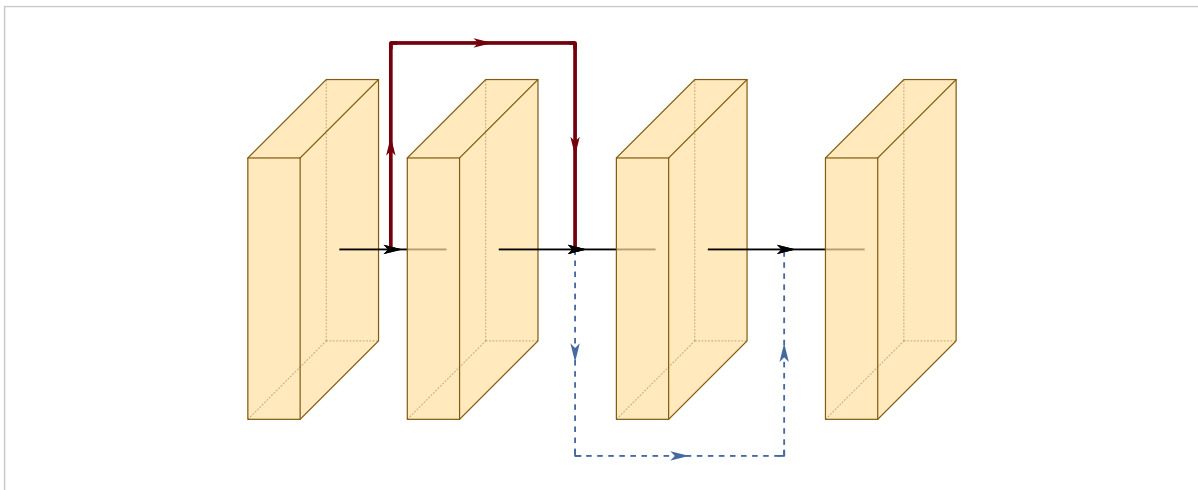
Any Typst dash pattern, e.g. "dashed", "dotted".

thickness

default 1

Multiplies the palette stroke width, so it composes with `stroke-thickness` rather than fighting it.

```
#draw-network(
  (
    conv(name: "a"), conv(name: "b"),
    conv(name: "c"), conv(name: "d"),
  ),
  connections: (
    connection(from: "a", to: "c", color: rgb("#73000A"), thickness: 2),
    connection(from: "b", to: "d", color: rgb("#466A9F"), dash: "dashed"),
  ),
)
```



Arrowheads take the line colour, since a coloured line with black arrowheads reads as a bug rather than a choice.

10.6 Routes in the legend

legend

default none

Adds a line-sample legend entry rather than a colour swatch, since what distinguishes a route is its stroke and not a fill. Routes sharing a name appear once.

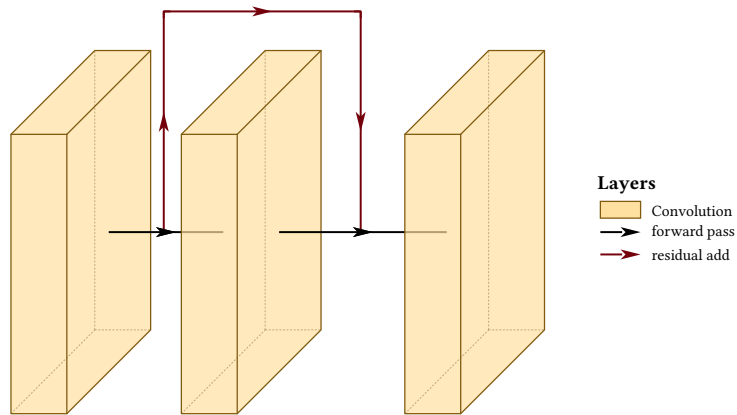
main-legend

default none

draw-network

Names the automatic axis arrows. Without it a legend can explain every skip in a figure and say nothing about the arrows carrying the forward pass.

```
#draw-network(
  (conv(name: "a"), conv(name: "b"), conv(name: "c")),
  connections: (
    connection(from: "a", to: "c", color: rgb("#73000A")),
    legend: "residual add"),
  ),
  show-legend: true,
  main-legend: "forward pass",
)
```



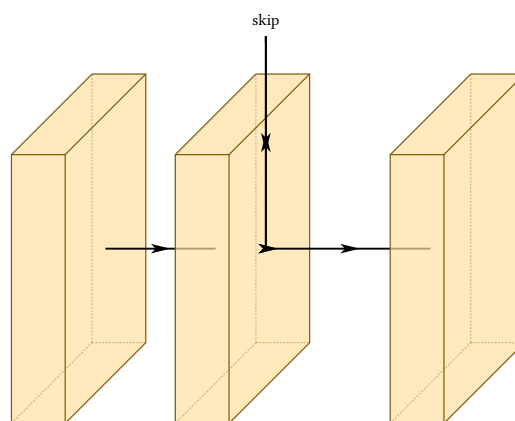
10.7 Labelling a route

label

default none

Text on the route.

```
#draw-network(
  (conv(name: "a"), conv(), conv(name: "c")),
  connections: (connection(from: "a", to: "c", label: "skip")),
)
```



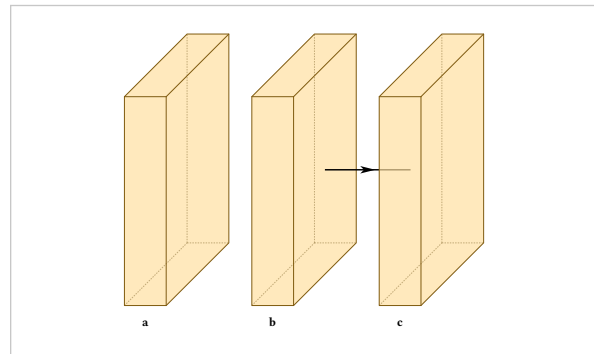
10.8 Suppressing an axis arrow

show-connection

default true; false on input

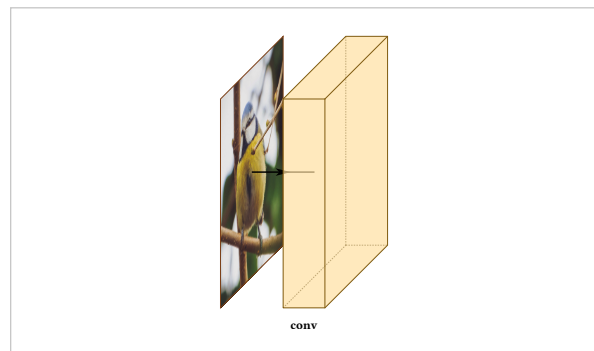
Whether to draw the axis arrow **after** this layer.

```
#draw-network((
  // no arrow after this one
  conv(label: "a", show-connection: false),
  conv(label: "b"),
  conv(label: "c"),
))
```



The input layer is the one that defaults to `false`, which is why an image usually sits detached. Set it `true` to join it up:

```
// input defaults to no outgoing arrow; turn it on
#draw-network((
  input(image: "default", show-connection: true),
  conv(label: "conv", offset: 1.5),
))
```



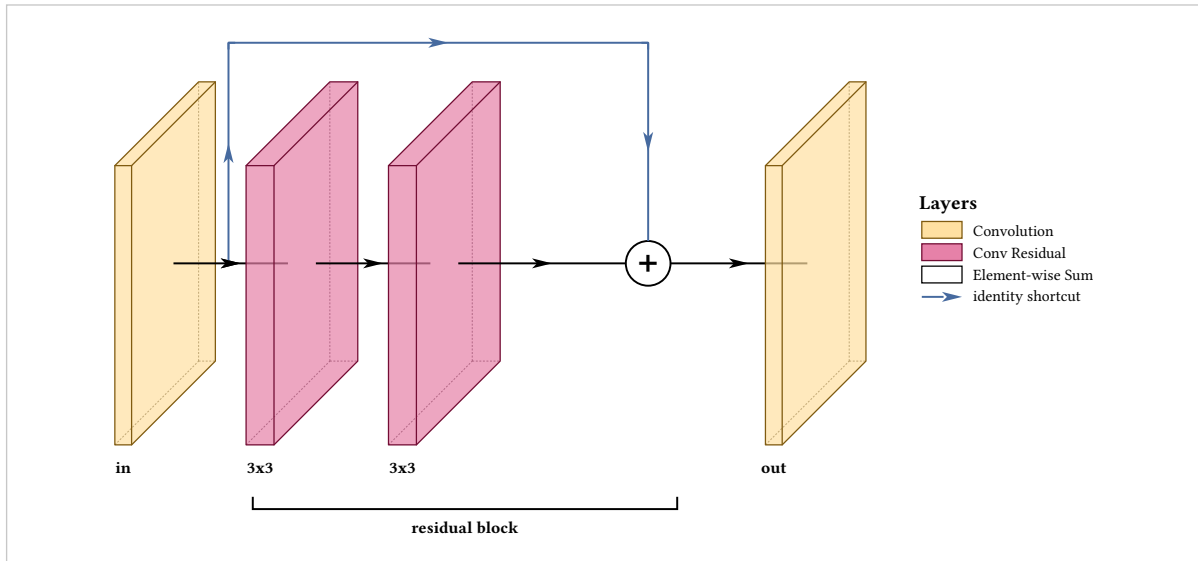
10.9 Two composites

A residual block is a sum node plus one route:

```

#draw-network(
(
  conv(name: "in", label: "in", widths: (0.3,)),
  convres(name: "c1", label: "3x3", widths: (0.5,)),
  convres(name: "c2", label: "3x3", widths: (0.5,)),
  sum(name: "add"),
  conv(name: "out", label: "out", widths: (0.3,)),
),
connections: (
  connection(from: "in", to: "add", color: rgb("#466A9F"),
    legend: "identity shortcut"),
),
groups: (group(from: "c1", to: "add", label: "residual block")),
show-legend: true,
)

```

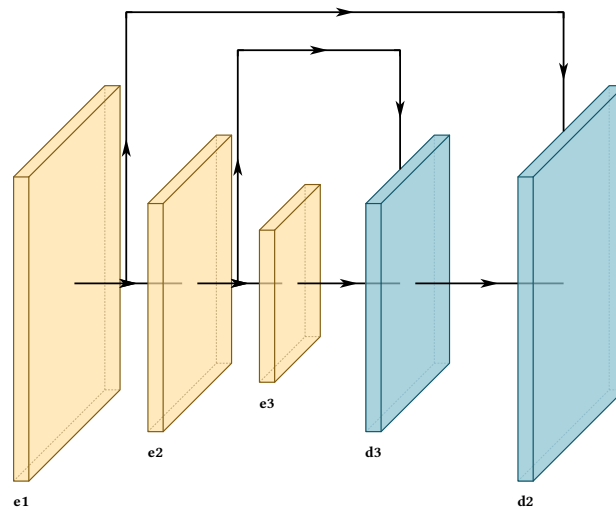


An encoder-decoder is a size pyramid plus two routes arriving on their targets:

```

#let enc = (widths: (0.3,))
#draw-network(
  (
    conv(name: "e1", label: "e1", height: 6, depth: 6, ..enc),
    conv(name: "e2", label: "e2", height: 4.5, depth: 4.5, ..enc),
    conv(name: "e3", label: "e3", height: 3, depth: 3, ..enc),
    deconv(name: "d3", label: "d3", height: 4.5, depth: 4.5),
    deconv(name: "d2", label: "d2", height: 6, depth: 6),
  ),
  connections: (
    connection(from: "e2", to: "d3", touch-layer: true),
    connection(from: "e1", to: "d2", touch-layer: true),
  ),
)

```



Groups and repeats

Backbone, neck and head are the phrases anyone uses out loud to explain one of these figures. `groups` draws them, as labelled brackets under a span of layers.

from default required

First layer of the span, by name.

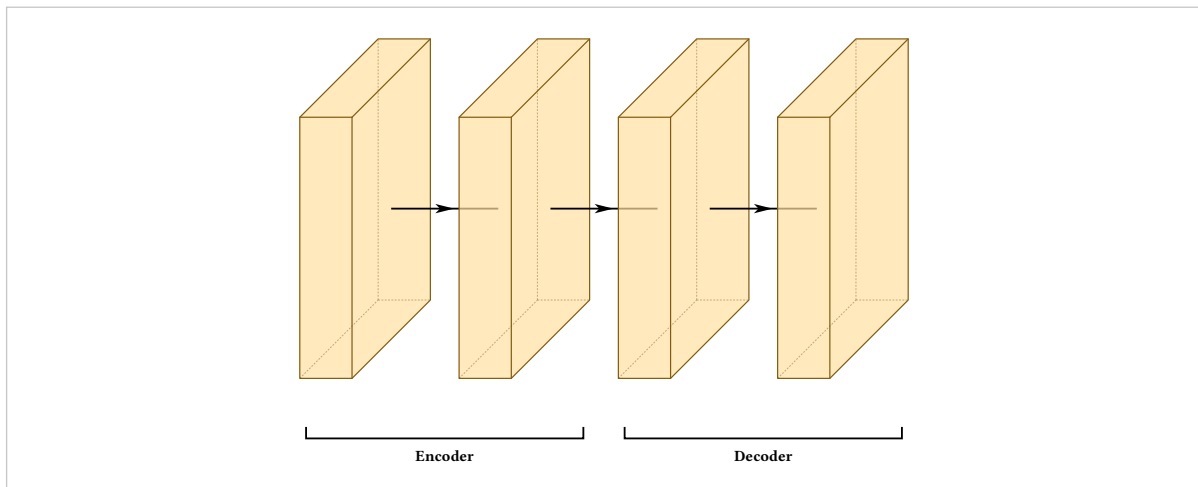
to default required

Last layer of the span, inclusive. May be the same layer.

label default none

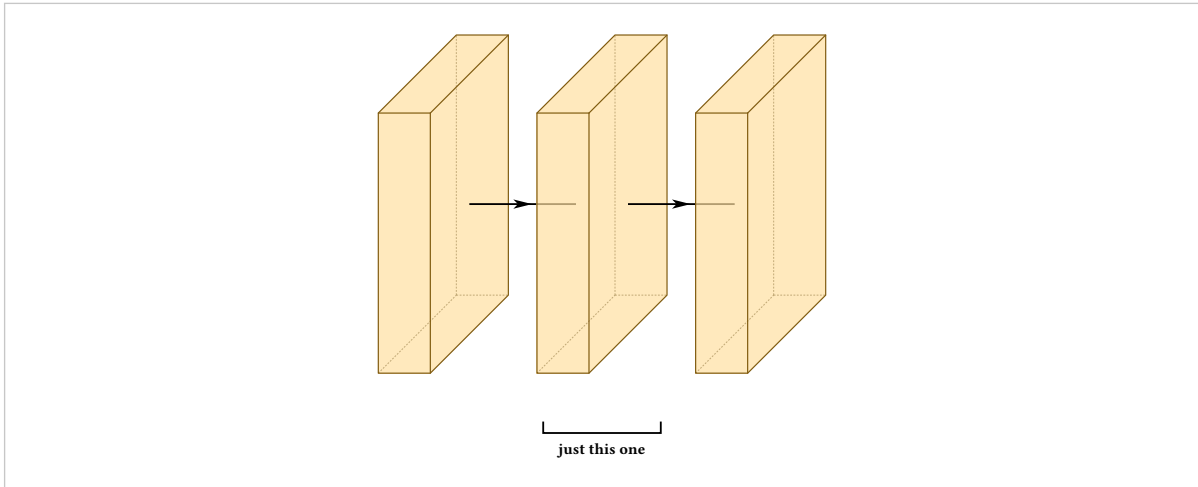
Bracket text.

```
#draw-network(
  (
    conv(name: "e1"), conv(name: "e2"),
    conv(name: "d1"), conv(name: "d2"),
  ),
  groups: (
    group(from: "e1", to: "e2", label: "Encoder"),
    group(from: "d1", to: "d2", label: "Decoder"),
  ),
)
```



A group of one is allowed and common, since a head is often a single block:

```
// from and to may be the same layer
#draw-network(
  (conv(name: "a"), conv(name: "b"), conv(name: "c")),
  groups: (group(from: "b", to: "b", label: "just this one"),),
)
```



The bracket covers the drawn footprint rather than the front faces, so it sits under the whole block including its lean, and its ends are inset slightly so two adjacent groups read as two rather than as one continuous rule.

11.1 Tinting and stacking

color

default grey

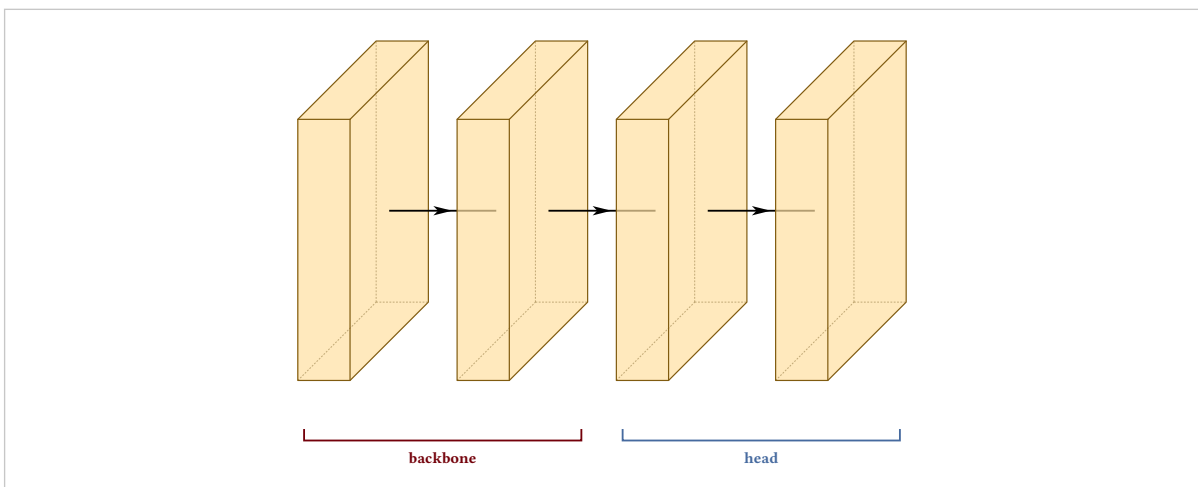
Tints one bracket.

offset

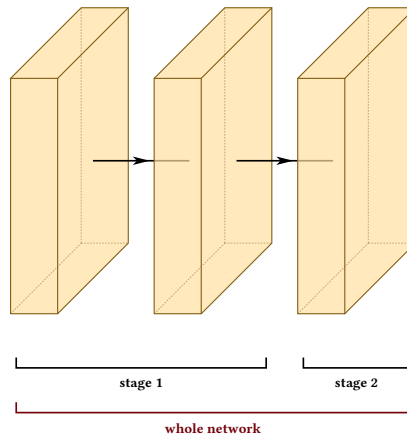
default 1.15

Pushes the bracket further down, which is how you stack a group that encloses other groups onto its own row.

```
#draw-network(
  (conv(name: "a"), conv(name: "b"), conv(name: "c"), conv(name: "d")),
  groups: (
    group(from: "a", to: "b", label: "backbone", color: rgb("#73000A")),
    group(from: "c", to: "d", label: "head", color: rgb("#466A9F")),
  ),
)
```

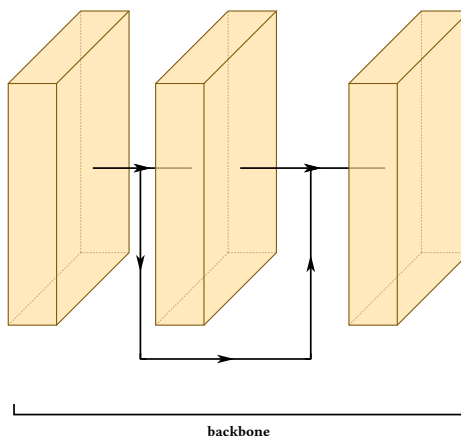


```
#draw-network(
  (conv(name: "a"), conv(name: "b"), conv(name: "c")),
  groups: (
    group(from: "a", to: "b", label: "stage 1"),
    group(from: "c", to: "c", label: "stage 2"),
    group(from: "a", to: "c", label: "whole network",
      offset: 2.1, color: rgb("#73000A")),
  ),
)
```



Brackets are placed below everything else in the figure, including a route running underneath the stack, so the two never collide:

```
// Brackets sit below routes that run underneath the stack
#draw-network(
  (conv(name: "a"), conv(name: "b"), conv(name: "c")),
  connections: (connection(from: "a", to: "c", mode: "flat")),
  groups: (group(from: "a", to: "c", label: "backbone")),
)
```



11.2 Repeated blocks

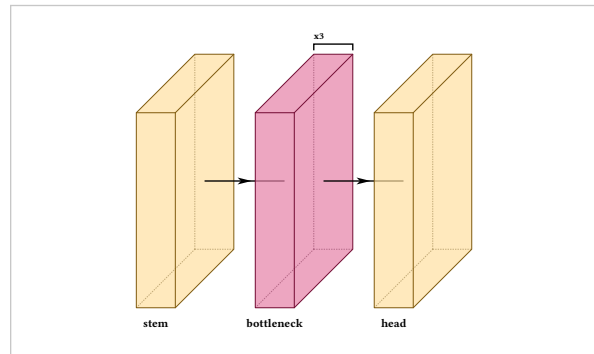
Depth-scaled models stack the same block several times. Writing that as extra entries in widths makes it **look** repeated, but the package has no idea it is: it cannot label the repeat or bracket it, and the figure claims one wide block rather than several.

repeat

default 1

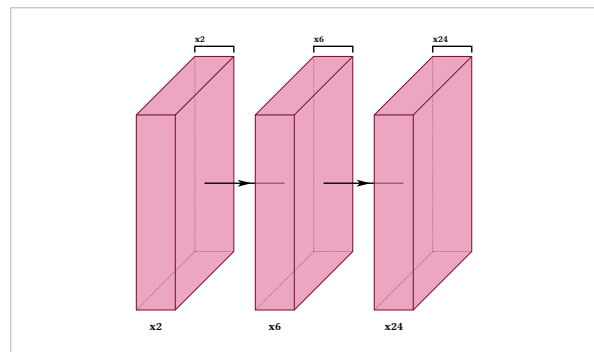
Marks the block as repeated in series. It is drawn once, with a bracket above it carrying the count. Ignored on pool, unpool and sum.

```
#draw-network((
  conv(label: "stem"),
  convres(label: "bottleneck", repeat: 3),
  conv(label: "head"),
))
```



The count costs no horizontal space, so it stays legible at any depth:

```
#draw-network((
  convres(label: "x2", repeat: 2),
  convres(label: "x6", repeat: 6),
  convres(label: "x24", repeat: 24),
))
```

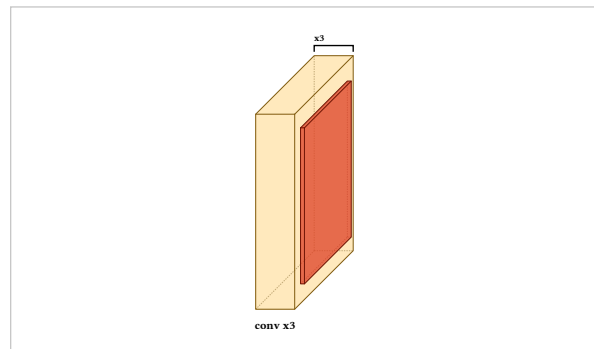


Ghosted copies were tried first and rejected. An outline behind the block reads as an empty box rather than as another one of the same block, and drawing N of them is either misleading about the count or unreadable once N is large. A bracket states the count instead of depicting it.

The pool pitfall

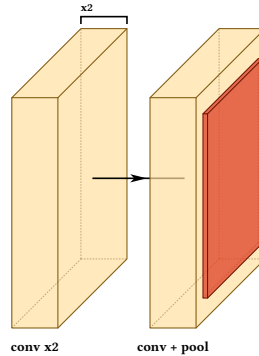
repeat describes one layer entry, not a run of them. Putting it on a convolution that has a pool attached reads as though the pool repeats too:

```
// Reads as though the pool repeats too
#draw-network((
  conv(label: "conv x3", repeat: 3),
  pool(),
))
```



Split it instead. Let the repeat cover the plain convolutions, and draw the last one on its own with the pool attached:

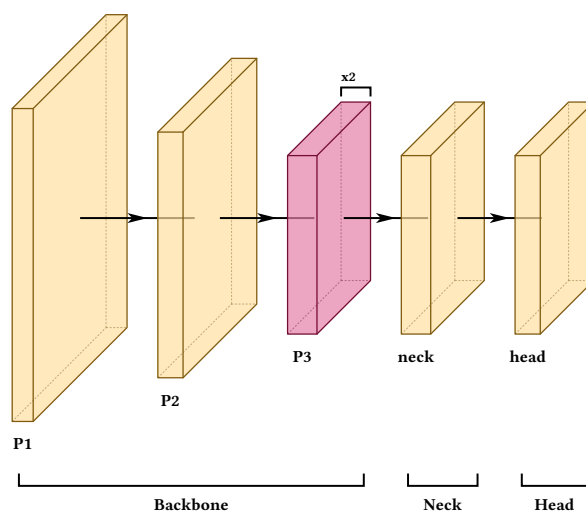
```
// Split it: two repeat, the third carries the pool
#draw-network((
  conv(label: "conv x2", repeat: 2),
  conv(label: "conv + pool", offset: 2),
  pool(),
))
```



For the same reason a repeated **pair**, such as attention-then-MLP, cannot be expressed with repeat at all. Section 15.5 shows what to do instead.

11.3 A composite

```
#draw-network(
  (
    conv(name: "s1", label: "P1", shape: (32, 160, 160)),
    conv(name: "s2", label: "P2", shape: (64, 80, 80)),
    convres(name: "s3", label: "P3", shape: (128, 40, 40), repeat: 2),
    conv(name: "n1", label: "neck", shape: (128, 40, 40)),
    conv(name: "h1", label: "head", shape: (64, 40, 40)),
  ),
  groups: (
    group(from: "s1", to: "s3", label: "Backbone"),
    group(from: "n1", to: "n1", label: "Neck"),
    group(from: "h1", to: "h1", label: "Head"),
  ),
)
```



Parallel branches

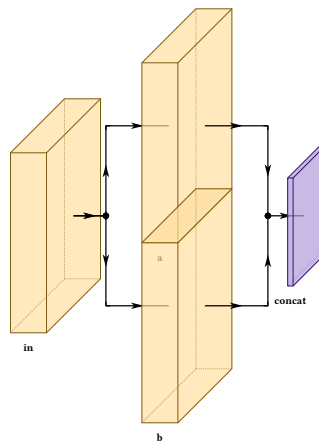
`draw-network` advances one cursor along one axis, so anything genuinely parallel — two detection heads, the two paths inside a CSP block, a two-stream fusion — would have to be collapsed into a single block with arrows pointed at it. A branch entry draws it as it is.

branches

default required

An array of layer arrays, one per parallel path. Each is walked exactly as the trunk is, so anything that works on the trunk works inside a branch.

```
#draw-network((
  conv(label: "in"),
  branch(spread: 5, branches: (
    (conv(label: "a"),),
    (conv(label: "b"),),
  )),
  concat(label: "concat"),
))
```



12.1 Separation and count

spread

default 6

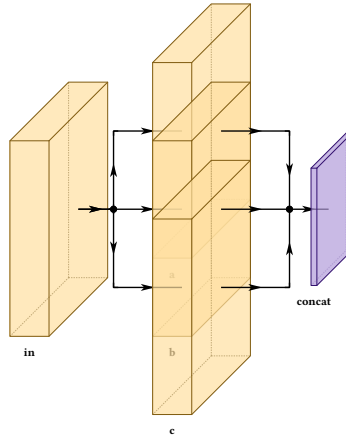
Separation between paths, centred on the trunk. An odd count puts one path on the trunk line; an even count leaves it empty.

lead

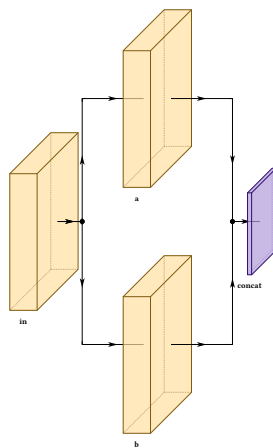
default 2.0

How far the fan-out and rejoin arrows run.

```
// An odd count puts one branch on the trunk line
#draw-network((
  conv(label: "in"),
  branch(spread: 4, branches: (
    (conv(label: "a")),
    (conv(label: "b")),
    (conv(label: "c")),
  )),
  concat(label: "concat"),
))
```

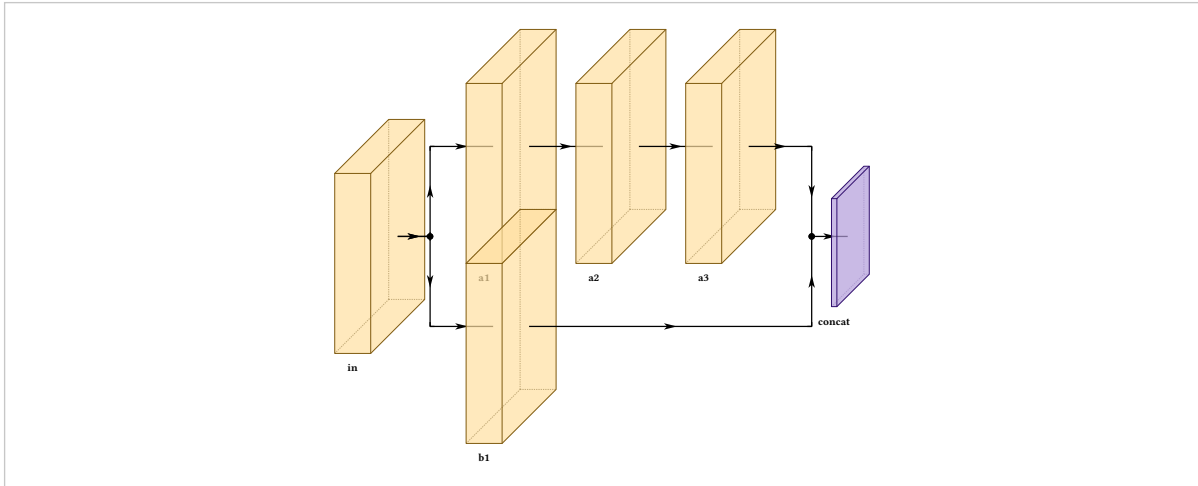


```
#draw-network((
  conv(label: "in"),
  branch(spread: 9, lead: 3, branches: (
    (conv(label: "a")),
    (conv(label: "b")),
  )),
  concat(label: "concat"),
))
```



Branches need not be the same length. The rejoin waits for the longest rather than cutting the others short:

```
// The rejoin waits for the longest branch
#draw-network((
  conv(label: "in"),
  branch(spread: 5, branches: (
    (conv(label: "a1"), conv(label: "a2"), conv(label: "a3")),
    (conv(label: "b1"),),
  )),
  concat(label: "concat"),
))
```



A filled dot marks each point where flow divides or meets, and where a tooth leaves a spine that passes through. Merging routes terminate at the dot, and a single arrow leaves it for the next block.

12.2 Stacking along the projection

spread-mode

default "vertical"

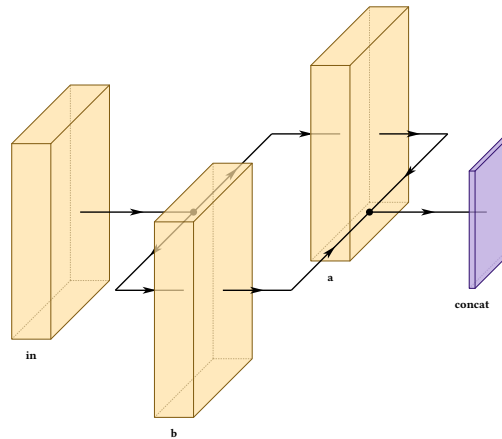
"depth" stacks the paths along the projection's 45-degree axis instead of straight up, so they read as parallel copies sitting behind one another.

rejoin-lead

default the value of lead

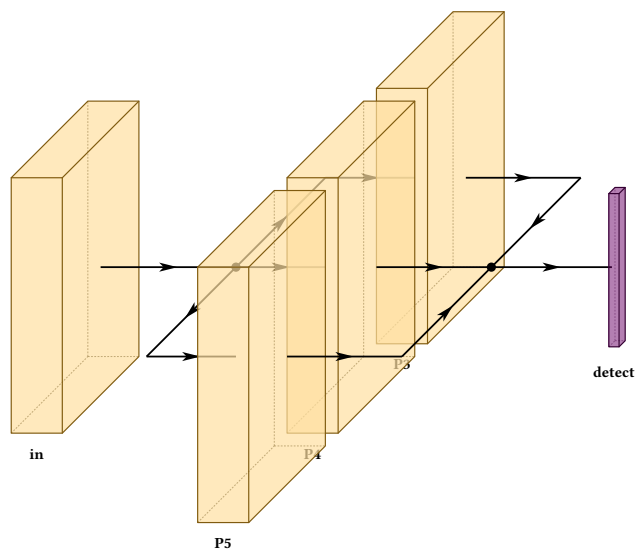
Widens the rejoin side alone. Depth mode often wants this, since its return spine descends past each block's lower-right corner, which is exactly where diagonal dimension labels sit.


```
#draw-network((
  conv(label: "in"),
  branch(spread: 4, spread-mode: "depth", branches: (
    (conv(label: "a")),
    (conv(label: "b")),
  )),
  concat(label: "concat"),
))
```



The fan-out and rejoin become two parallel spines with horizontal teeth — a parallelogram, rather than the mirrored hexagon a vertical split produces. Neither 45 is a free angle: it is the one the projection already uses. This is the natural way to draw a set of detection heads:

```
#draw-network((
  conv(label: "in"),
  branch(
    spread: 3.5, spread-mode: "depth", rejoin-lead: 3,
    branches: (
      (conv(label: "P3")),
      (conv(label: "P4")),
      (conv(label: "P5")),
    ),
  ),
  output(label: "detect"),
))
```



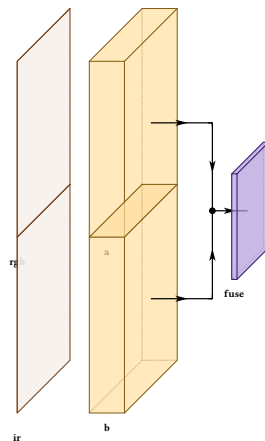
12.3 Open at one end

open

default none

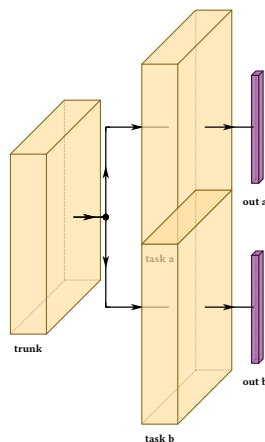
"start" draws no fan-out, "end" no rejoin. A branch with nothing before it is open at the start by definition. A branch open at the start is how a multi-input network begins:

```
// Nothing before it: a two-input network
#draw-network((
  branch(spread: 5, branches: (
    (input(label: "rgb"), conv(label: "a")),
    (input(label: "ir"), conv(label: "b")),
  )),
  concat(label: "fuse"),
))
```



Open at the end, one trunk fans out into independent outputs:

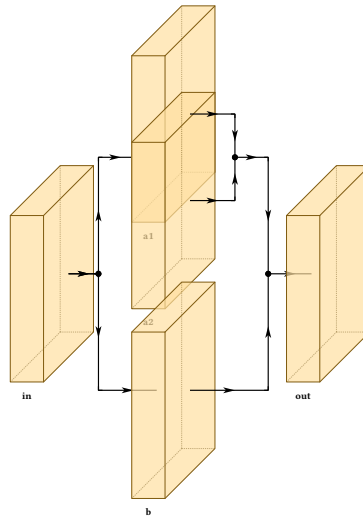
```
#draw-network((
  conv(label: "trunk"),
  branch(spread: 5, open: "end", branches: (
    (conv(label: "task a"), output(label: "out a")),
    (conv(label: "task b"), output(label: "out b")),
  )),
))
```



12.4 Nesting

A branch may contain branches, since walking a branch is the same operation as walking the trunk.

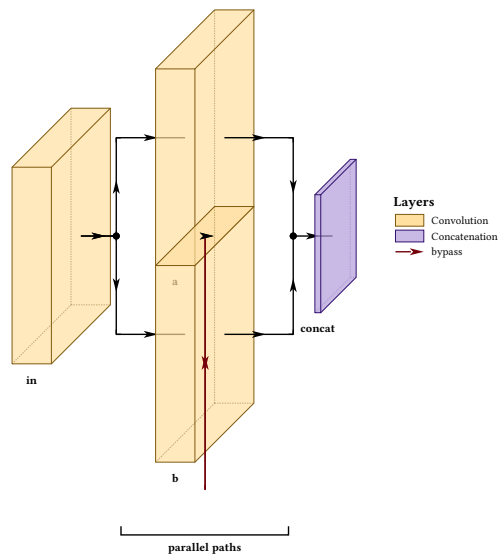
```
#draw-network((
  conv(label: "in"),
  branch(spread: 7, branches: (
    (
      branch(spread: 2.6, lead: 1.2, branches: (
        (conv(label: "a1")),
        (conv(label: "a2")),
      )),
    (conv(label: "b")),
  )),
  conv(label: "out"),
))
```



12.5 Branches are not a separate world

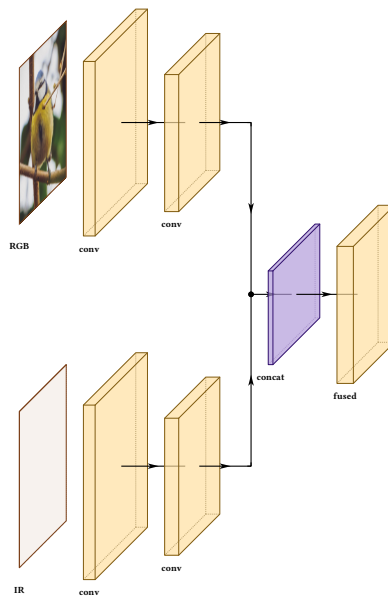
Named layers inside a branch join the same table the trunk uses, so connections, groups and automatic lane ranking all reach into them. A group naming any layer inside a branch widens to the branch's whole drawn extent, plumbing included.

```
// Named layers inside a branch are ordinary targets
#draw-network(
  (
    conv(name: "in", label: "in"),
    branch(spread: 5, branches: (
      (conv(name: "a", label: "a"),),
      (conv(name: "b", label: "b"),),
    )),
    concat(name: "cat", label: "concat"),
  ),
  connections: (
    connection(from: "in", to: "cat", mode: "flat",
      color: rgb("#73000A"), legend: "bypass"),
  ),
  groups: (group(from: "a", to: "b", label: "parallel paths"),),
  show-legend: true,
)
```



12.6 Two composites

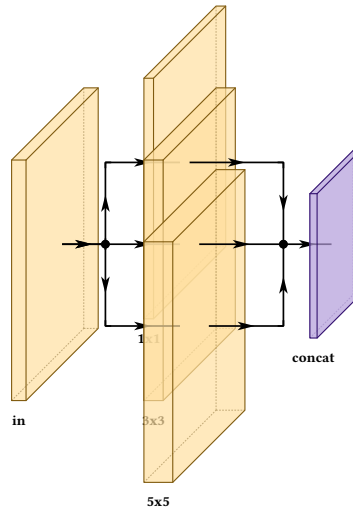
```
#draw-network((  
  branch(spread: 11, branches: (  
    (  
      input(label: "RGB", image: "default"),  
      conv(name: "r1", label: "conv", shape: (32, 160, 160)),  
      conv(name: "r2", label: "conv", shape: (64, 80, 80)),  
    ),  
    (  
      input(label: "IR"),  
      conv(name: "i1", label: "conv", shape: (32, 160, 160)),  
      conv(name: "i2", label: "conv", shape: (64, 80, 80)),  
    ),  
  )),  
  concat(label: "concat", depth: 5),  
  conv(label: "fused", shape: (128, 80, 80)),  
))
```



```

#draw-network((
  conv(label: "in", widths: (0.3,)),
  branch(spread: 3.4, lead: 1.6, branches: (
    (conv(label: "1x1", widths: (0.2,)),),
    (conv(label: "3x3", widths: (0.4,)),),
    (conv(label: "5x5", widths: (0.6,)),),
  )),
  concat(label: "concat"),
))

```



The figure as a whole

13.1 Legends

show-legend

default false

draw-network

Collects every layer type used into a legend, in order of first appearance.

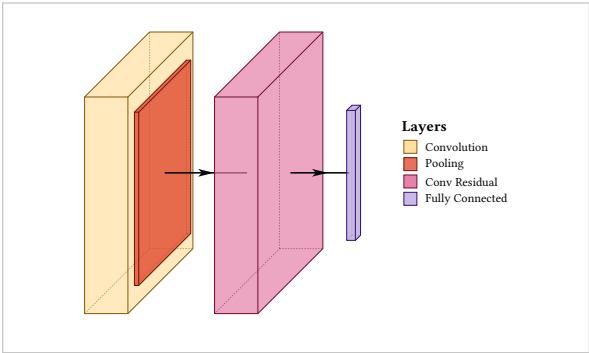
legend-title

default "Layers"

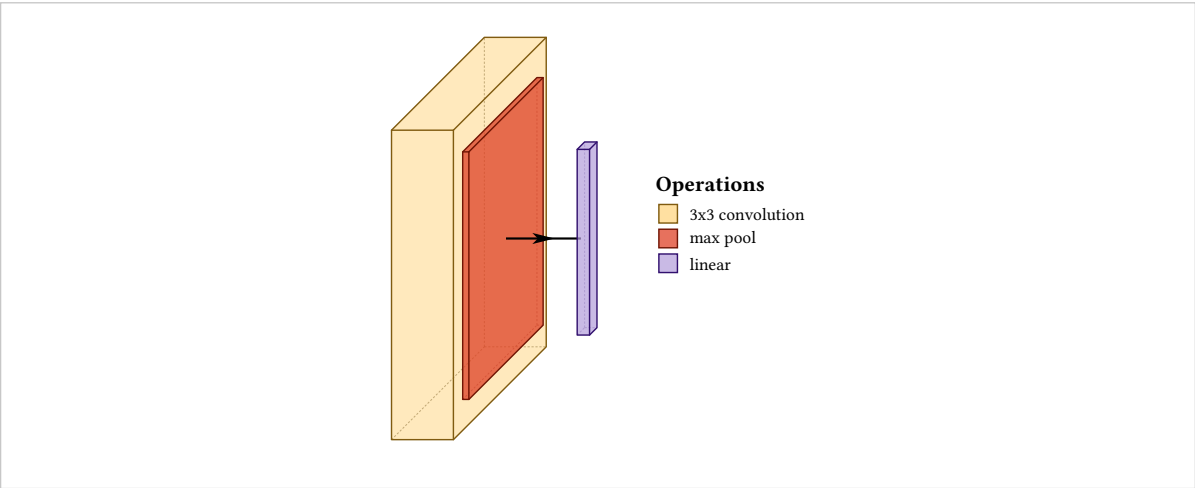
draw-network

Heading of the box.

```
#draw-network(  
  (conv(), pool(), convres(), fc()),  
  show-legend: true,  
)
```

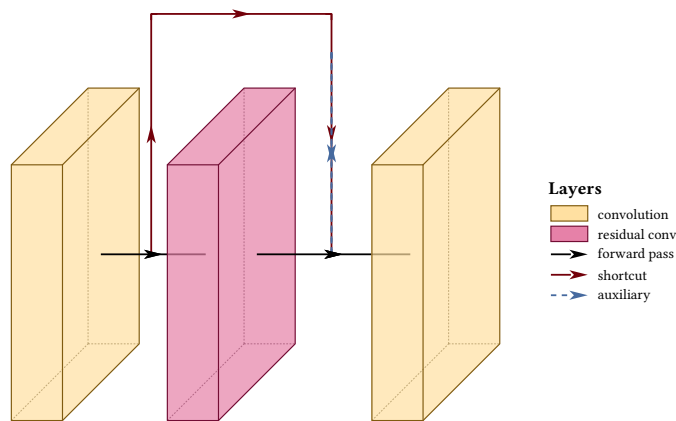


```
#draw-network(  
  (  
    conv(legend: "3x3 convolution"),  
    pool(legend: "max pool"),  
    fc(legend: "linear"),  
  ),  
  show-legend: true,  
  legend-title: "Operations",  
)
```



Layer swatches and route samples share one box. The sample column widens when any route entry is present, so a dash pattern has room to read, and the layer swatches widen with it so nothing floats away from its label:

```
// Layer swatches and route samples in one legend
#draw-network(
  (
    conv(name: "a", legend: "convolution"),
    convres(name: "b", legend: "residual conv"),
    conv(name: "c"),
  ),
  connections: (
    connection(from: "a", to: "c", color: rgb("#73000A"), legend: "shortcut"),
    connection(from: "b", to: "c", color: rgb("#466A9F"), dash: "dashed",
      legend: "auxiliary"),
  ),
  show-legend: true,
  main-legend: "forward pass",
)
```



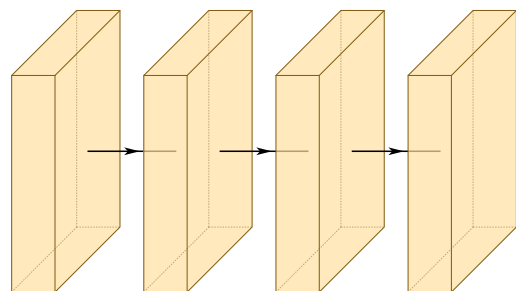
13.2 Fitting the page scale

default 100%

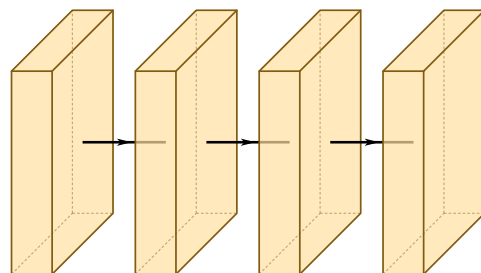
draw-network

Shrinks or grows the whole figure, text included.

```
#draw-network(
  (conv(), conv(), conv(), conv()),
  scale: 100%,
)
```



```
#draw-network(
  (conv(), conv(), conv(), conv()),
  scale: 55%,
)
```



A long network is wider than a page at its natural size, and scale is the first thing to reach for. If it makes the text too small to read, the next thing to try is label-orient: "diagonal" (section 6.5), which usually buys back enough horizontal space to raise the scale again.

13.3 Line weight

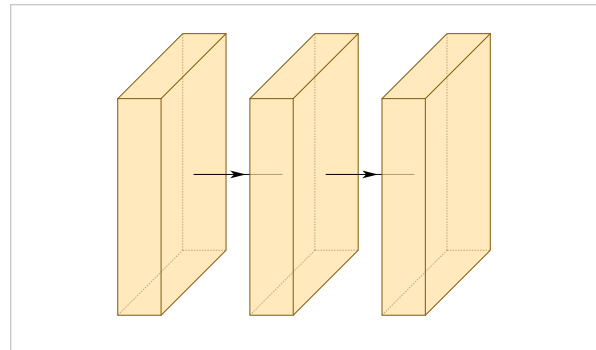
stroke-thickness

default 1

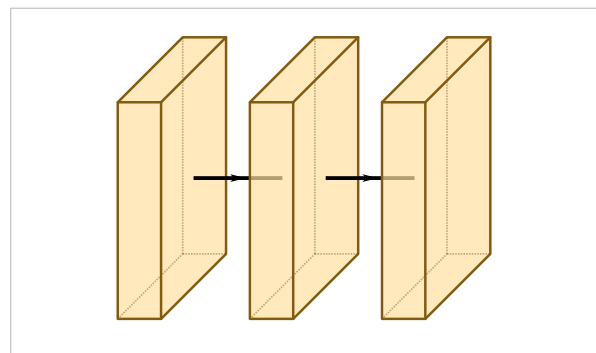
draw-network

Multiplies every stroke in the figure. Connection thickness multiplies on top of this. Worth raising when a figure is reduced for a two-column layout, where hairlines start dropping out at print resolution:

```
#draw-network(  
  (conv(), conv(), conv()),  
  stroke-thickness: 0.5,  
)
```



```
#draw-network(  
  (conv(), conv(), conv()),  
  stroke-thickness: 2.5,  
)
```



13.4 The projection

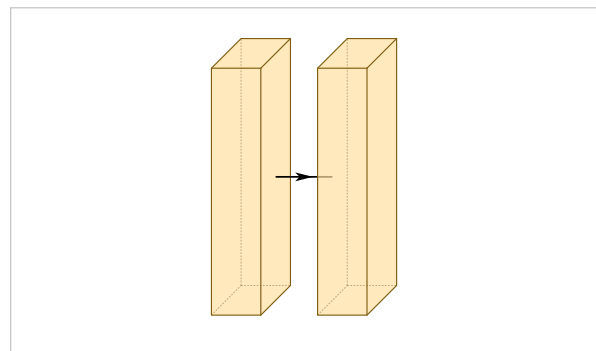
depth-multiplier

default 0.3

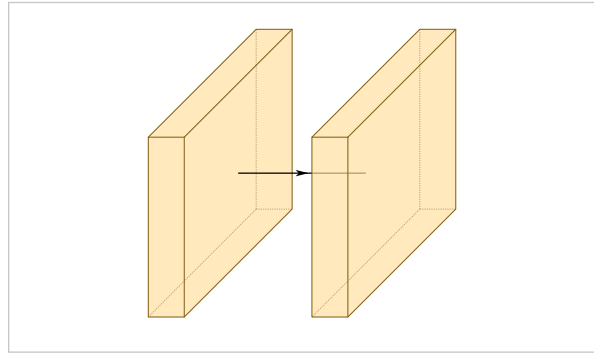
draw-network

How far blocks lean, which is the strength of the isometric projection. It also changes what depth-shear and min-clear-offset return, so pass the same value to those.

```
#draw-network(  
  (conv(depth: 5), conv(depth: 5)),  
  depth-multiplier: 0.12,  
)
```



```
#draw-network(  
  (conv(depth: 5), conv(depth: 5)),  
  depth-multiplier: 0.6,  
)
```



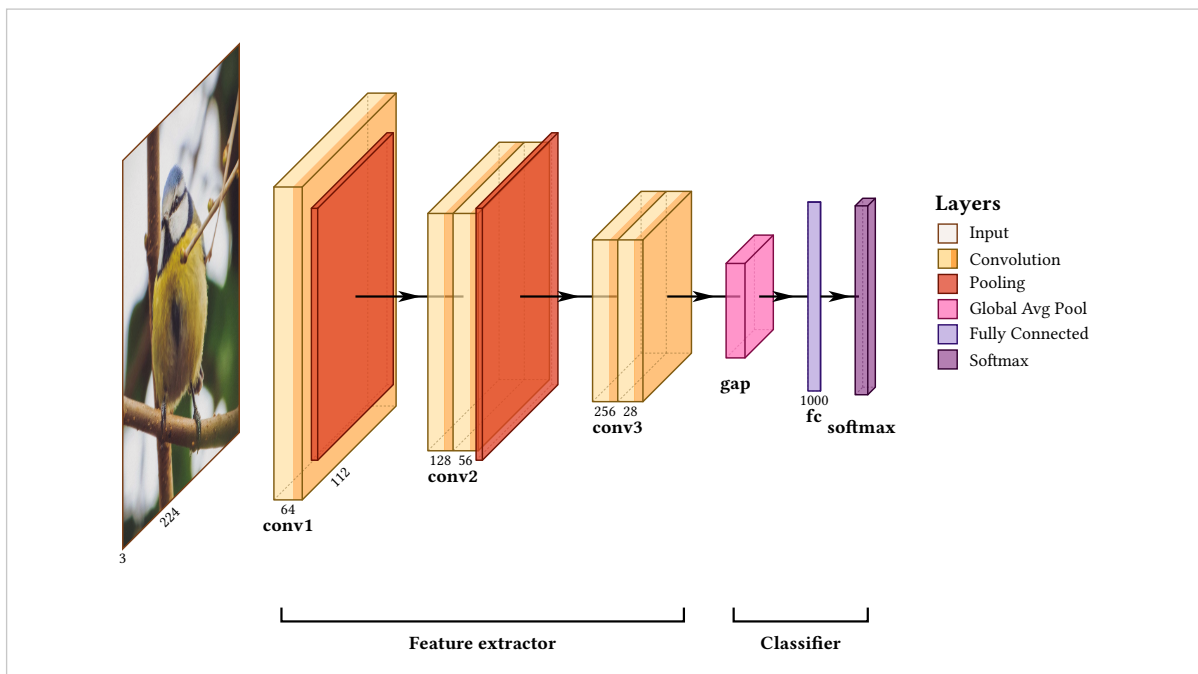
Patterns

Everything here is built only from what the preceding chapters covered. These are not architectures to copy so much as shapes to recognise: most figures are one of them with different labels.

14.1 A classifier

Types, shapes and two groups.

```
#draw-network(
(
  input(name: "in", image: "default", shape: (3, 224, 224),
    channels: (3, 224)),
  conv(name: "c1", label: "conv1", shape: (64, 112, 112),
    channels: (64, 112)),
  pool(),
  conv(name: "c2", label: "conv2", shape: (128, 56, 56),
    channels: (128, 56), widths: (0.4, 0.4)),
  pool(),
  conv(name: "c3", label: "conv3", shape: (256, 28, 28),
    channels: (256, 28), widths: (0.4, 0.4)),
  gap(name: "g", label: "gap"),
  fc(name: "f", label: "fc", channels: (1000,), depth: 0),
  softmax(name: "s", label: "softmax"),
),
groups: (
  group(from: "c1", to: "c3", label: "Feature extractor"),
  group(from: "g", to: "s", label: "Classifier"),
),
show-legend: true,
show-relu: true,
)
```



14.2 An encoder-decoder

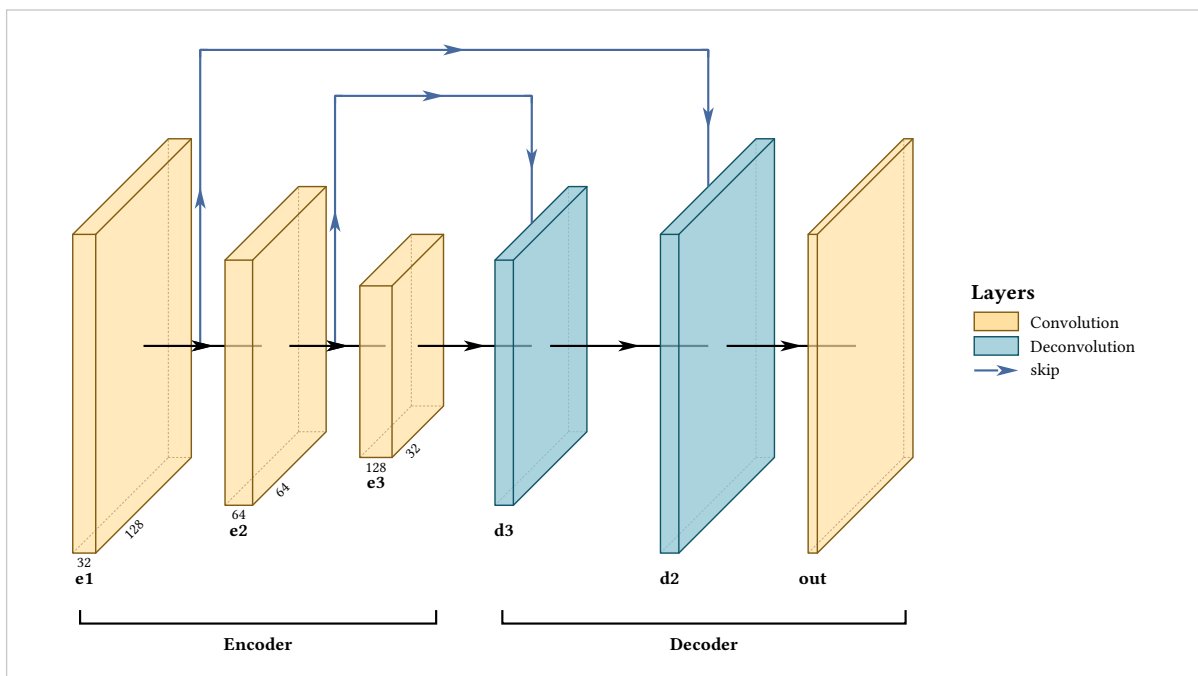
A size pyramid down and back up, with skips arriving on their targets. The two deconv blocks take their size from shape exactly as the encoder does, so the figure is symmetric because the model is, not because it was measured.

```

#let e(n, s) = conv(name: n, label: n, shape: s, channels: (s.at(0), s.at(1)))
#let d(n, s) = deconv(name: n, label: n, shape: s)

#draw-network(
  (
    e("e1", (32, 128, 128)),
    e("e2", (64, 64, 64)),
    e("e3", (128, 32, 32)),
    d("d3", (64, 64, 64)),
    d("d2", (32, 128, 128)),
    conv(name: "out", label: "out", shape: (1, 128, 128)),
  ),
  connections: (
    connection(from: "e2", to: "d3", touch-layer: true,
      color: rgb("#466A9F"), legend: "skip"),
    connection(from: "e1", to: "d2", touch-layer: true,
      color: rgb("#466A9F")),
  ),
  groups: (
    group(from: "e1", to: "e3", label: "Encoder"),
    group(from: "d3", to: "out", label: "Decoder"),
  ),
  show-legend: true,
)

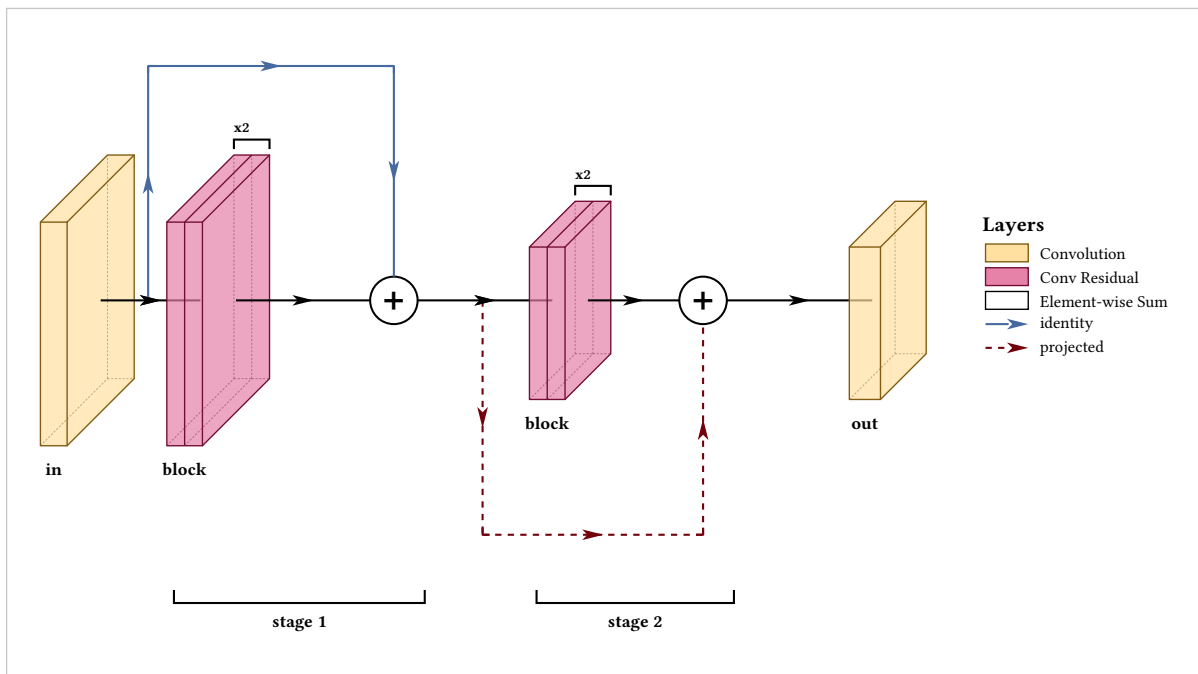
```



14.3 Residual stages

repeat for the blocks inside a stage, a sum node for each add, and two route colours to separate an identity shortcut from a projected one.

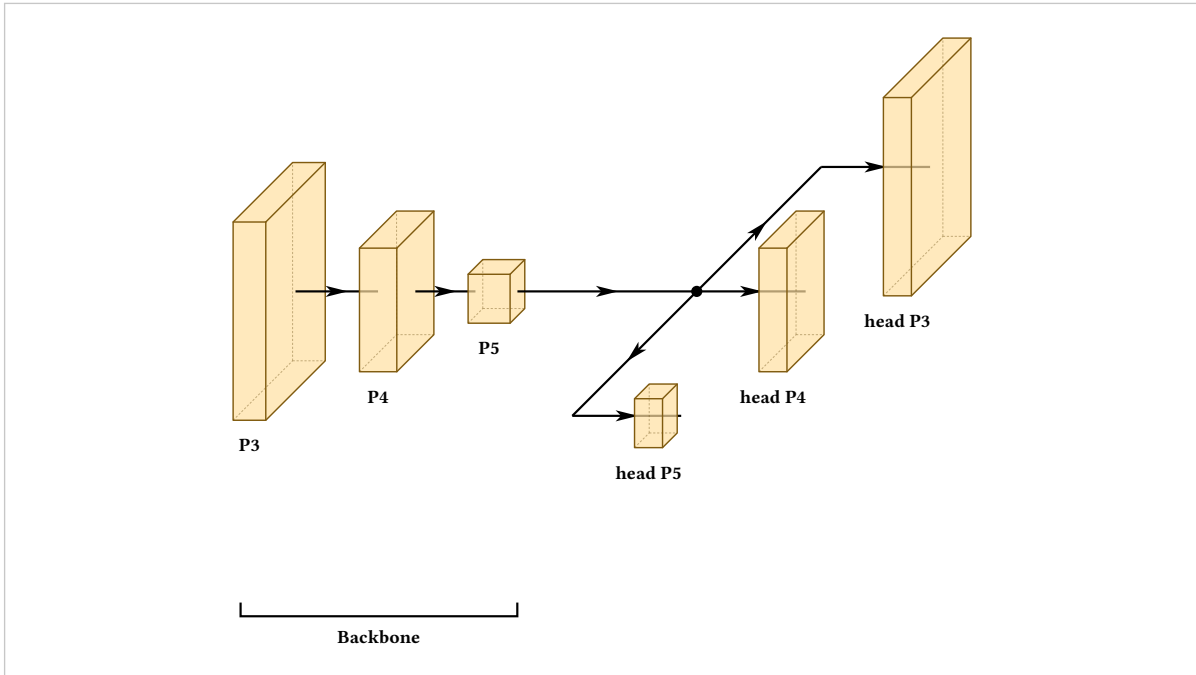
```
#draw-network(
(
  conv(name: "in", label: "in", shape: (64, 56, 56)),
  convres(name: "b1", label: "block", shape: (64, 56, 56),
    widths: (0.3, 0.3), repeat: 2),
  sum(name: "a1"),
  convres(name: "b2", label: "block", shape: (128, 28, 28),
    widths: (0.3, 0.3), repeat: 2),
  sum(name: "a2"),
  conv(name: "out", label: "out", shape: (128, 28, 28)),
),
connections: (
  connection(from: "in", to: "a1", color: rgb("#466A9F"),
    legend: "identity"),
  connection(from: "a1", to: "a2", color: rgb("#73000A"), dash: "dashed",
    legend: "projected"),
),
groups: (
  group(from: "b1", to: "a1", label: "stage 1"),
  group(from: "b2", to: "a2", label: "stage 2"),
),
show-legend: true,
)
```



14.4 A multi-scale head

A depth-mode branch, open at the end, is what a detector's heads look like.

```
#draw-network(
(
  conv(name: "p3", label: "P3", shape: (128, 40, 40)),
  conv(name: "p4", label: "P4", shape: (256, 20, 20)),
  conv(name: "p5", label: "P5", shape: (512, 10, 10)),
  branch(spread: 4, spread-mode: "depth", rejoin-lead: 3.2,
    open: "end", branches: (
      (conv(name: "h3", label: "head P3", shape: (64, 40, 40))),
      (conv(name: "h4", label: "head P4", shape: (64, 20, 20))),
      (conv(name: "h5", label: "head P5", shape: (64, 10, 10))),
    )),
  ),
  groups: (group(from: "p3", to: "p5", label: "Backbone")),
)
```

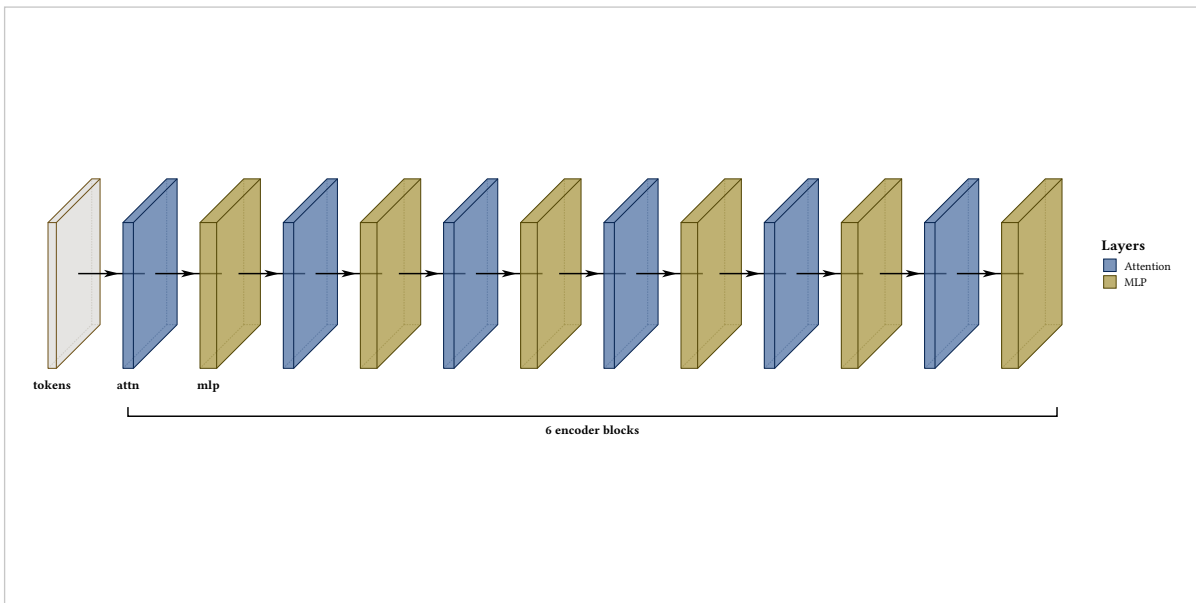


14.5 A repeated pair

repeat cannot express a repeated **pair** (section 11.3), so build the list instead. Twelve blocks are written once, and the count lives in the `range` rather than in the figure:

```
// Twelve identical blocks, written once
#let block(i) = (
  custom(name: "a" + str(i), label: if i == 1 { "attn" },
    fill: rgb("#466A9F"), width: 0.25, height: 3.5, depth: 3.5,
    legend: "Attention"),
  custom(name: "m" + str(i), label: if i == 1 { "mlp" },
    fill: rgb("#A49137"), width: 0.4, height: 3.5, depth: 3.5,
    legend: "MLP"),
)

#draw-network(
  (custom(name: "in", label: "tokens", width: 0.2,
    height: 3.5, depth: 3.5),)
  + range(1, 7).map(block).flatten(),
  groups: (group(from: "a1", to: "m6", label: "6 encoder blocks")),
  show-legend: true,
)
```



This is the general answer to any figure that is getting long. The layer list is an array and a constructor is a function; anything Typst can do to those, you can do to a figure.

Every option

The tables below are generated from the package itself rather than written out here, so an option cannot exist without appearing in this chapter.

15.1 Options by layer type

Type	Options
branch	branches, lead, name, open, rejoin-lead, spread, spread-mode
concat	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
conv	bandfill, channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, show-relu, widths, xlabel, ylabel, zlabel
convres	bandfill, channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, show-relu, widths, xlabel, ylabel, zlabel
convsoftmax	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
custom	bandfill, channels, connection-label, depth, fill, height, image, input-style, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, show-relu, width, widths, xlabel, ylabel, zlabel
deconv	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
fc	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection
gap	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection
input	channels, connection-label, depth, fill, height, image, input-style, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
output	channels, classes, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
pool	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection
softmax	channels, classes, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection, width
sum	channels, connection-label, depth, fill, height, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, radius, show-connection, stroke, symbol
unpool	channels, connection-label, depth, fill, height, image, label, label-anchor, label-angle, label-dx, label-dy, label-orient, legend, name, offset, opacity, repeat, shape, show-connection

15.2 Connection options

arrive-offset · clearance · color · dash · from · label · layers · legend · mode · opacity · pos · thickness · to · touch-layer · type

15.3 Group options

color · from · label · offset · to

15.4 Arguments to draw-network

```
#draw-network(  
  layers,  
  connections: (),  
  groups: (),  
  palette: "warm",  
  show-legend: false,  
  legend-title: "Layers",  
  main-legend: none,  
  scale: 100%,  
  stroke-thickness: 1,  
  depth-multiplier: 0.3,  
  lane-unit: 0.75,  
  shape-scale: (spatial: (1.2, -3.2), channels: (0.075, 0.0)),  
  auto-gap: 0.55,  
  show-relu: false,  
)
```

15.5 Also exported

```
depth-shear(depth, depth-multiplier: 0.3)  
min-clear-offset(depth, depth-multiplier: 0.3)
```

Plus a constructor per layer type, and `connection(..)` and `group(..)`.

15.6 Coverage

Every option exported by the package is described somewhere above.

Errors

Every message the package raises, and what causes it.

unknown layer option "X" on layer N (type "T")

x is not an option of that type. The message lists what is, and suggests the nearest match. A correctly spelled option belonging to a **different** type gives this too — widths on a pool, width on a conv.

unknown layer type "X" on layer N

Not one of the fourteen. See section 16.1.

layer N is a plain dictionary

Every layer comes from a constructor. Write (type: "conv", ..) as conv(..); a dictionary of options spreads into it as conv(..opts).

connection N refers to "X", which is not the `name` of any layer

Either the target has no name, or the reference is misspelled. Groups give the same message.

connection N has no `from`

A route names both layers it runs between.

`branches` on layer N must be an array of layer arrays

A common slip is passing one layer list rather than a list of them. Two branches of one layer each is ((conv()),), (conv(),)).

branch open must be "start" or "end"

label-orient must be "horizontal", "diagonal" or "vertical"

Use either label-orient or label-angle, not both

The right anchor for an arbitrary rotation depends on where you want the text, so the package will not guess.

shape must be (channels, height, width)

Three numbers, in that order.

array index out of bounds (index: 1, len: 1)

Raised from inside src/lib.typ rather than as a message of the package's own. Almost always channels with more entries than the layer has bands, plus one. Either add matching widths entries or remove channel entries.

unexpected argument: X

Typst's own message, from a constructor given an option that does not exist. It is raised before the package sees the call, which is one reason to prefer constructors.

Where to go next

Complete architectures are in `examples/` in the repository, and their rendered output in `gallery/`. They are ordinary Typst files, meant to be read and copied:

examples/networks/ AlexNet, LeNet-5, VGG16, VGG19, ResNet18, U-Net, FCN-8 and YOLO26-n, plus five RGB-IR fusion variants of the last one.
examples/features/ every predefined layer type side by side in both palettes, and a figure exercising every feature at once.
examples/imported/ figures generated from a traced PyTorch model rather than written by hand, using `tools/import_model.py`.

The YOLO26-n example is the one to read if you want to see the automatic machinery carrying a whole figure. Every block states its tensor shape, every gap is automatic, every lane height is automatic, and the three detection heads are a depth-mode branch. Nothing in it is sized or positioned by hand.

This manual is itself plain text. `doc/manual.typ` and the 125 files in `doc/examples/` are the whole of it, each example a standalone figure that compiles on its own. That source, rather than this PDF, is what to point a language model at.